Intelligent Systems

01.09.2026

03.09.2026

Universidad Espíritu Santo

Week 1

Introducción a la inteligencia artificial: definición, historia, casos de uso y ética

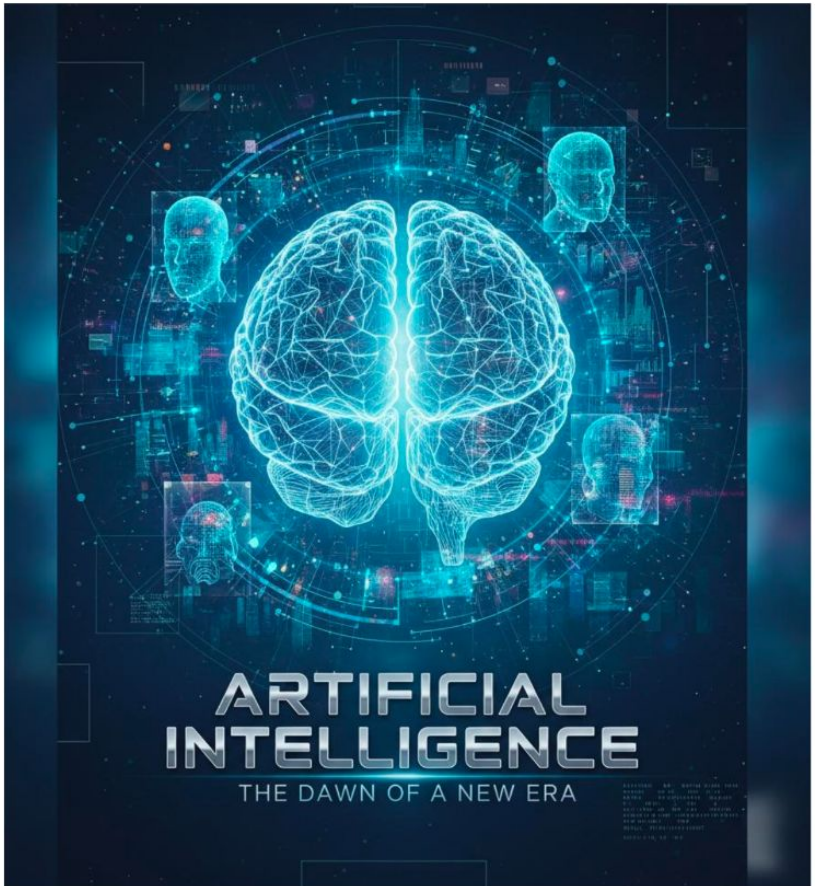
01.09.2026 - 03.09.2026

Topics covered in this course

1. Introduction to artificial intelligence
2. Machine learning
3. Supervised learning and unsupervised learning
4. Neural networks

Learning outcomes

- Define the concept of artificial intelligence
- Describe the main events that occurred before the rise of artificial intelligence


ARTIFICIAL
INTELLIGENCE
THE DAWN OF A NEW ERA

What is artificial intelligence?



https://youtu.be/NbEbs6I3eLw

Artificial intelligence - definition



John McCarthy
(2007)

" It is the science and engineering of making intelligent machines, especially intelligent computer programs. It is related to the similar task of using computers to understand human intelligence, but AI does not have to confine itself to methods that are biologically observable."

¿Es IA o no? Ejemplos cotidianos

EJEMPLO

Según McCarthy, la IA es hacer máquinas que realizan tareas que requieren inteligencia (percibir, aprender, decidir). Una pista útil: **¿el sistema aprende de datos o solo sigue reglas fijas?**

Sistema	¿IA?	¿Por qué?
Calculadora que suma 2 + 2	No	Ejecuta una regla fija programada; no aprende ni decide.
Alarma que suena a las 6:00	No	Es una instrucción fija: «a esta hora, suena».
Filtro de spam de Gmail	Sí	Aprendió de millones de correos marcados como spam o no spam.
Desbloqueo facial del celular	Sí	Reconoce tu rostro aunque cambie la luz o lleves gafas (visión por computadora).
Google Maps: «ruta más rápida»	Sí	Predice el tráfico con datos históricos y en tiempo real.
Netflix: «Porque viste…»	Sí	Recomienda a partir de patrones de usuarios con gustos parecidos.

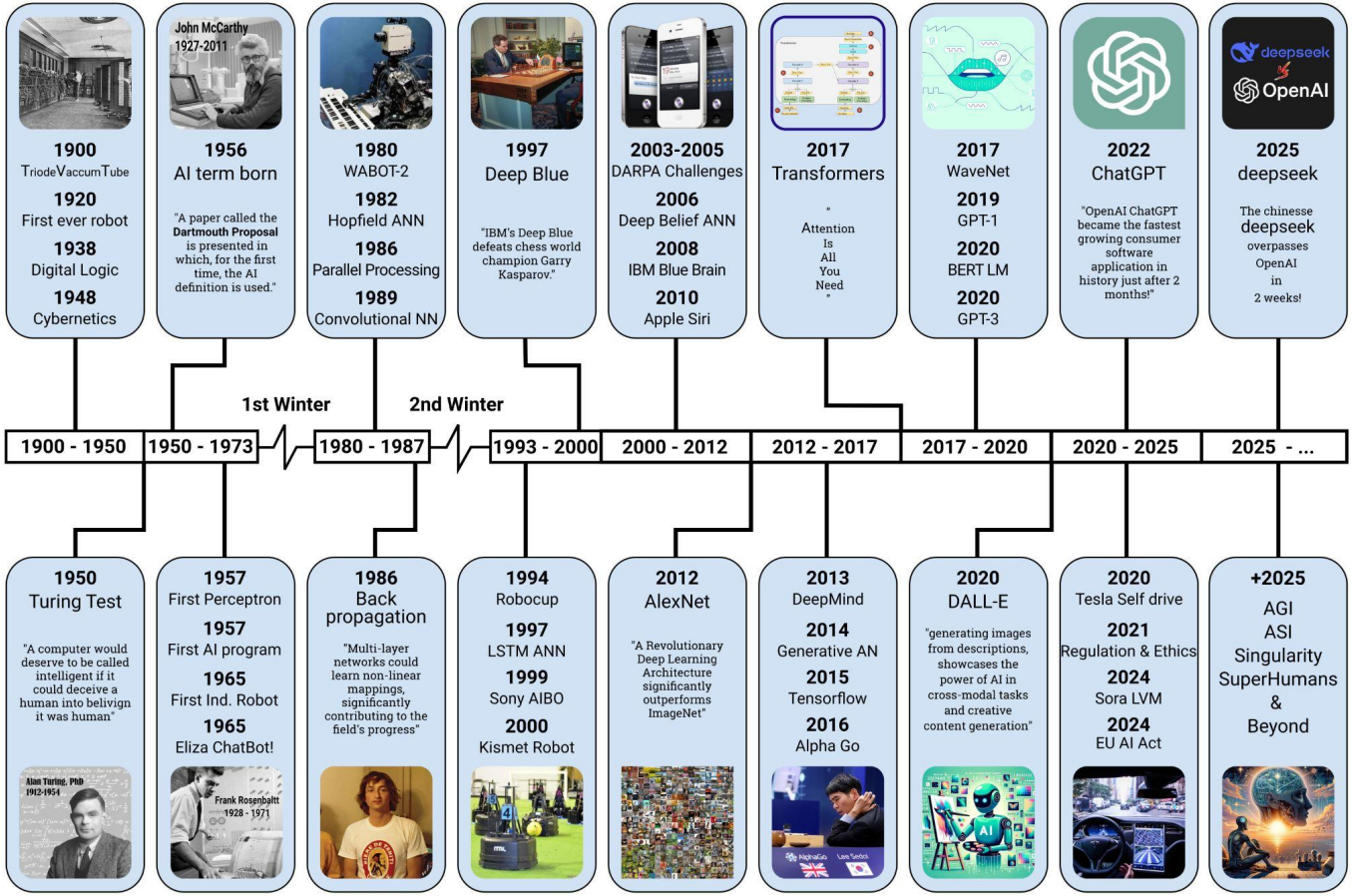


Image source: AI History Timeline – Wikipedia

AI Winter

First AI Winter (1970s):

- Caused by unmet expectations from early AI systems.
- Limited computing power hindered progress.
- Funding cuts due to slow advancements.

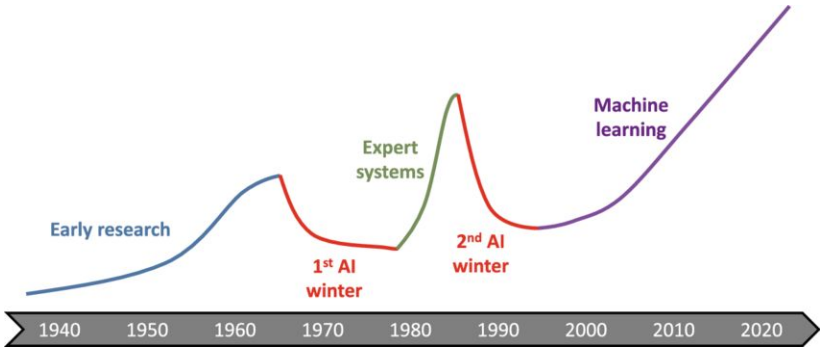


Image source: Colliot, Olivier. (2023). A Non-technical Introduction to Machine Learning.

Second AI Winter (late 1980s–1990s):

- Decline of expert systems, which were expensive, inflexible, and hard to maintain.
- High costs and low scalability of AI applications.
- Reduced investment and skepticism in AI research.

¿Por qué llegaron los inviernos de la IA?

EJEMPLO

Grandes promesas → Resultados pobres → Decepción → Recorte de fondos = «invierno»

1966 · Informe ALPAC (EE. UU.)

Se prometía traducción automática casi perfecta. El informe concluyó que era más lenta, menos precisa y más cara que un traductor humano → se recortó la financiación.

1973 · Informe Lighthill (R. Unido)

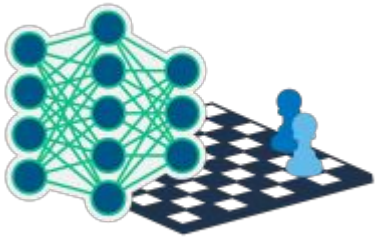
Concluyó que la IA no había cumplido sus objetivos fuera de problemas «de juguete» → recortes en la investigación de IA en universidades británicas.

Fines de los 80 · Sistemas expertos

Funcionaban con miles de reglas escritas a mano (p. ej., XCON de DEC). Eran caros de mantener y no se adaptaban a casos nuevos → en 1987 colapsa el mercado de las máquinas Lisp.

Analogía: como una película con un tráiler espectacular que decepciona en el cine: la gente deja de comprar boletos (los inversionistas dejan de financiar) hasta que llega una mejora real.

Artificial intelligence - evolution



1950-1980
Artificial intelligence



1980-2010
Machine learning



2010-present
Deep learning

Image source: SAS Company

Un mismo problema en tres épocas: detectar spam

EJEMPLO

1950–1980 · IA con reglas

Un programador escribe las reglas a mano:

```
if "GRATIS" in correo:
    marcar_spam()
```

Problema: el spammer escribe «GR4TIS» y la regla falla.

1980–2010 · Machine learning

Se le dan **miles de correos etiquetados** (spam / no spam). El algoritmo **aprende solo** qué palabras y patrones pesan más: «gratis» +0.8, «reunión» -0.6…

Mejora al recibir más ejemplos.

2010–hoy · Deep learning

Una red neuronal con muchas capas lee el correo completo y capta el **contexto**: detecta un engaño aunque no use ninguna palabra «sospechosa».

Necesita muchos datos y mucha capacidad de cómputo.

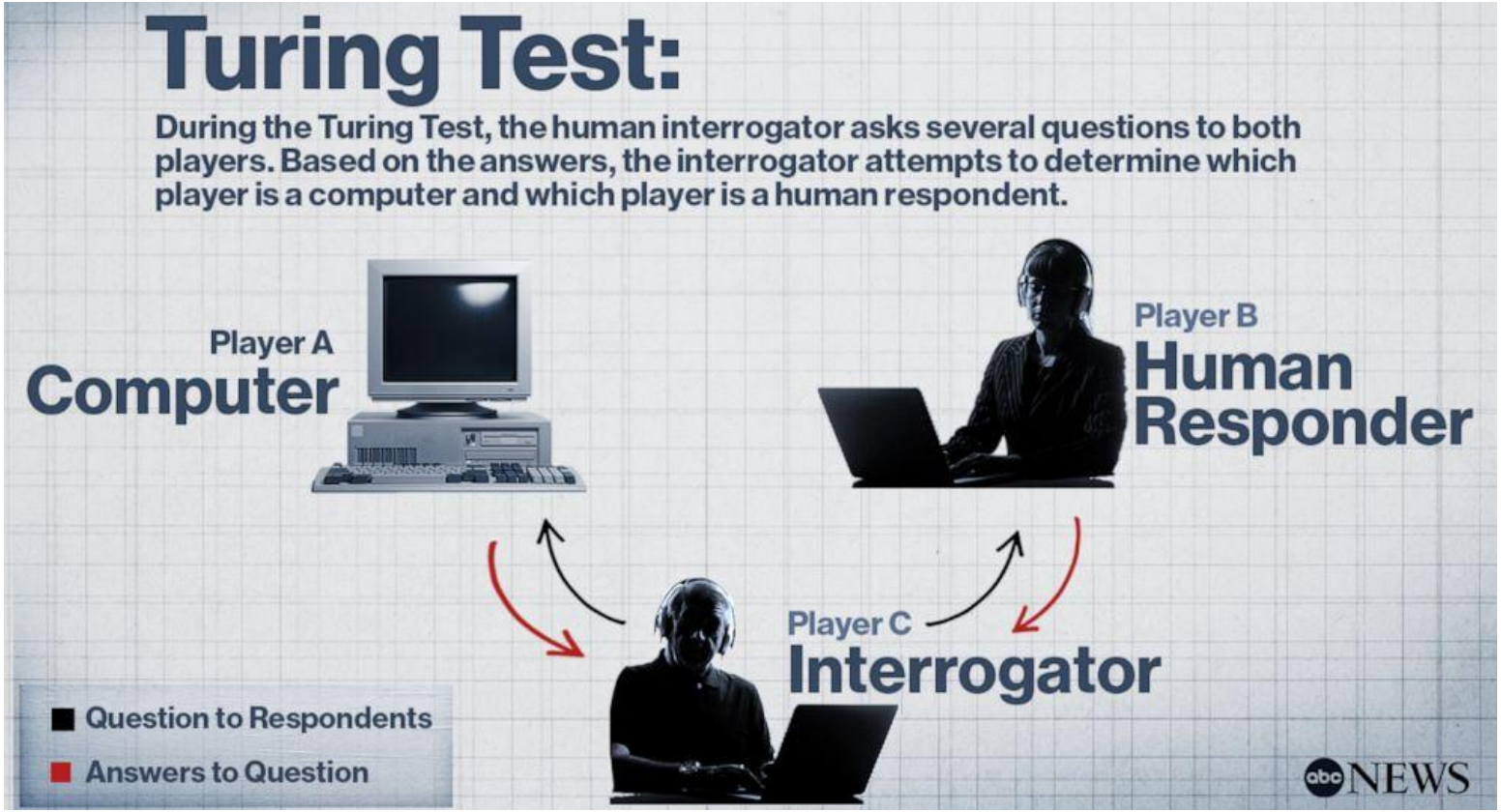
Cada etapa está **contenida** en la anterior: el deep learning es un tipo de machine learning, y el machine learning es una rama de la IA.

Alan Turing

- Considered one of the fathers of computer science and artificial intelligence.
- Creator of the Turing test (1950), a test that determines whether one is speaking to a machine or a person.



Image source: Biography


Turing Test:
During the Turing Test, the human interrogator asks several questions to both players. Based on the answers, the interrogator attempts to determine which player is a computer and which player is a human respondent.
Player A
Computer
Player B
Human Responder
Player C
Interrogator
Question to Respondents
Answers to Question
abc NEWS

La prueba de Turing en acción

EJEMPLO

El interrogador **C** chatea por escrito con **A** y **B** sin verlos. Uno es una computadora.

	A	B
C: ¿Qué desayunaste hoy?	Un bolón con café, como siempre.	Solo un café, iba tarde.
C: ¿Cuánto es 3 847 × 29?	(tarda unos segundos) Uf… ¿como 111 mil y algo?	Ni idea sin calculadora, jaja.
C: ¿Qué te da miedo?	Quedarme sin batería en el bus.	Las arañas.

A es la máquina: **finge dudar y equivocarse** como un humano (la respuesta exacta es 111 563).

¿Cuándo «pasa» la prueba?

Si C no logra distinguir quién es la máquina mejor que adivinando al azar.

Hoy

En un estudio de 2025 (UC San Diego), GPT-4.5, con instrucciones de actuar como persona, fue elegido como «el humano» el 73 % de las veces.

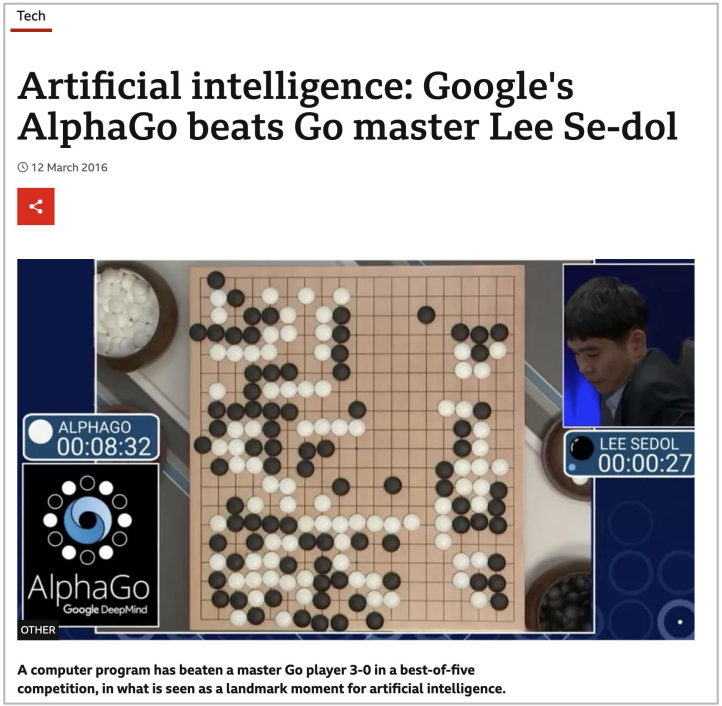
Pasar la prueba muestra que **imita** bien una conversación, no necesariamente que **entienda** (ver el argumento de la habitación china, Week 2).

Kasparov vs Deep Blue (1997)



Image source: kasparov.com

AlphaGo (2016)



Tech

Artificial intelligence: Google's AlphaGo beats Go master Lee Se-dol

12 March 2016

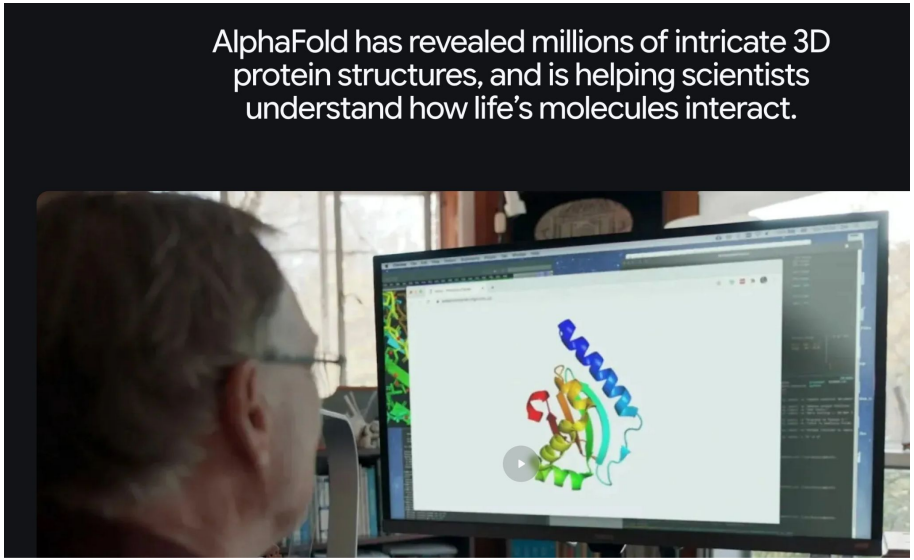
A computer program has beaten a master Go player 3-0 in a best-of-five competition, in what is seen as a landmark moment for artificial intelligence.

Image source: https://www.bbc.com/news/technology-35785875



https://youtu.be/WXuK6gekU1Y

Alphafold (2018)





https://youtu.be/d95J8yzvjbQ

Tres hitos, tres formas de «inteligencia»

EJEMPLO

Deep Blue (IBM, 1997)

Fuerza bruta + reglas de expertos. Evaluaba unos 200 millones de posiciones de ajedrez por segundo y elegía la mejor.

Solo sabía jugar ajedrez: no podía aprender otro juego.

AlphaGo (DeepMind, 2016)

El Go tiene más posiciones posibles que átomos en el universo observable: la fuerza bruta no sirve.

Aprendió de partidas humanas y jugando millones de partidas contra sí mismo. Venció 4-1 a Lee Sedol.

AlphaFold (DeepMind, 2018)

Predice la **forma 3D de una proteína** a partir de su secuencia de aminoácidos, un problema abierto por unos 50 años.

Sus creadores compartieron el Nobel de Química 2024.

La tendencia: de **reglas programadas por personas** (Deep Blue) a sistemas que **aprenden de los datos y de la experiencia** (AlphaGo, AlphaFold).

Generative AI

- a.k.a *gen AI*
- A type of artificial intelligence that can generate original content such as images, text, music and video.
- It relies on deep learning models to create gen AI applications.
- Some examples of generative AI systems include: StyleGAN, ChatGPT-3, GPT-4 and DALL-E.

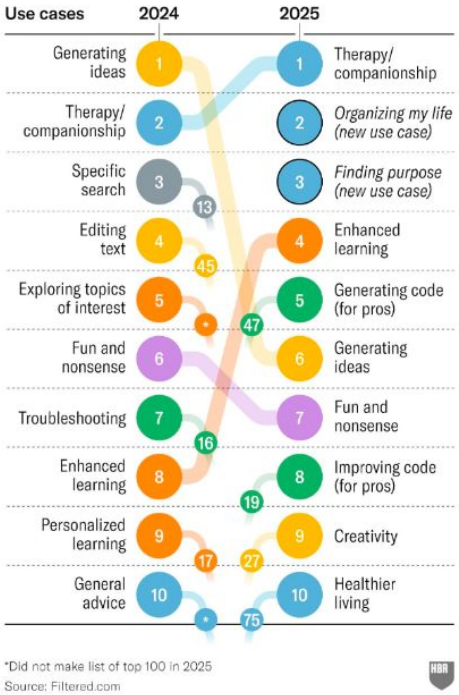
Generative AI - examples








Generative AI - use cases



IA tradicional vs. IA generativa

EJEMPLO

IA tradicional: clasifica o predice

Foto de un gato → Modelo → **«gato» (97 %)**

Elige una respuesta entre opciones que ya existen.

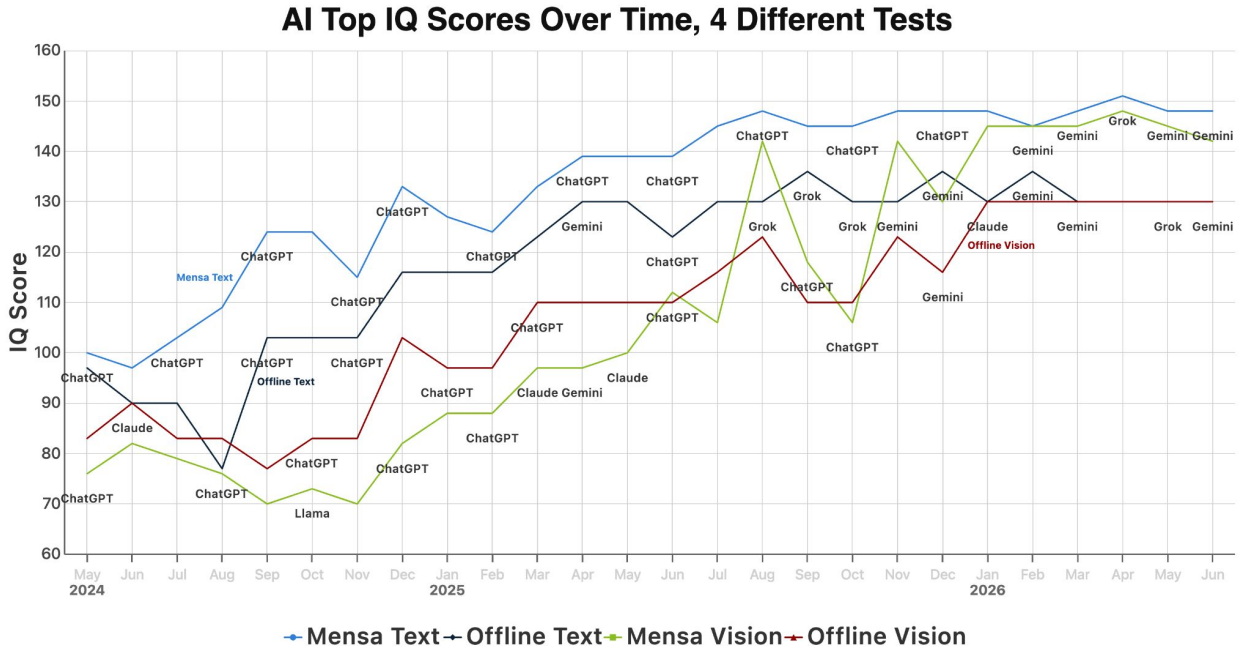
IA generativa: crea contenido nuevo

«Un gato astronauta en la Luna, estilo acuarela» → Modelo → **Imagen nueva**

Produce algo que nunca existió, siguiendo los patrones que aprendió.

Tipo de contenido	Ejemplo de uso
Texto	ChatGPT redacta un correo formal para pedir una prórroga.
Imagen	DALL-E crea la ilustración de una portada; StyleGAN genera rostros de personas que no existen.
Música	Se describe «salsa alegre sobre Guayaquil» y el sistema compone una canción.
Código	GitHub Copilot sugiere el resto de una función mientras escribes.

IQ Test Results for different verbal and vision AI solutions



Music composed by AI?



https://youtu.be/-PnLYNAWIWA

AI Commercial



https://youtu.be/4Rpi1ZMKy74

Humanoid Robot Sports Competition



https://youtu.be/2JA5xGteito

Use cases

- Crime prediction
- Traffic accident prediction
- Forest fire detection
- Online exam supervision (proctoring)

Use cases

- Text autocomplete
- CV scanning
- Dynamic pricing

Casos de uso: ¿qué entra y qué sale?

EJEMPLO

Predicción de crímenes

Historial de delitos por zona y hora → zonas con más riesgo para organizar patrullajes.

Accidentes de tránsito

Clima, hora, tráfico, historial → «riesgo alto en esta avenida hoy a las 18:00 con lluvia».

Incendios forestales

Imágenes de cámaras o satélites → alerta temprana al detectar humo.

Proctoring

Webcam y micrófono durante un examen → alerta: «otra persona detectada».

Autocompletado

«Nos vemos ma...» → sugiere «mañana».

Escaneo de CV

500 hojas de vida → ranking según experiencia y habilidades pedidas.

Precios dinámicos

Demanda, hora, lluvia → la tarifa de Uber sube cuando muchos piden auto a la vez.

Patrón común

Todos **aprenden de datos pasados** para predecir o decidir sobre casos nuevos.

Ethics of AI



https://youtu.be/1LyacmzB1Og

Ethical dilemmas of artificial intelligence

- Biases (gender, age, race)
- Decision making
- Privacy
- Property, intellectual rights
- Environmental impact

Dilemas éticos con ejemplos concretos

EJEMPLO

Dilema	Ejemplo
Sesgos	En 2018 se reportó que una herramienta de reclutamiento de Amazon penalizaba CVs con la palabra «women's»: aprendió de 10 años de contrataciones mayoritariamente masculinas.
Toma de decisiones	Un auto autónomo debe decidir en milisegundos entre frenar o desviarse. Si falla, ¿quién responde: el fabricante, el programador o el dueño?
Privacidad	Clearview AI recopiló miles de millones de fotos de redes sociales, sin permiso, para su sistema de reconocimiento facial.
Propiedad intelectual	Getty Images demandó en 2023 a Stability AI por usar millones de sus imágenes para entrenar un generador de imágenes.
Impacto ambiental	Entrenar y usar modelos grandes requiere centros de datos que consumen mucha electricidad y agua para refrigeración.

Un modelo **aprende lo que ve en los datos**: si los datos tienen sesgos, el modelo los repite (como Tay o Google Translate).

Microsoft Tay - robot



BBC NEWS MUNDO

Noticias América Latina Internacional Hay Festival Economía Ciencia Salud

Centroamérica Cuenta BBC Extra

Tay, la robot racista y xenófoba de Microsoft

Redacción
BBC Mundo

25 marzo 2016

A tan sólo un día de su lanzamiento, Tay comenzó a emitir comentarios e insultos racistas y xenófobos.

Se llamaba Tay y tardó más en aprender a mantener una conversación que en ser retirada del mercado.



Digital Innovation and Transformation – Harvard

Google translate

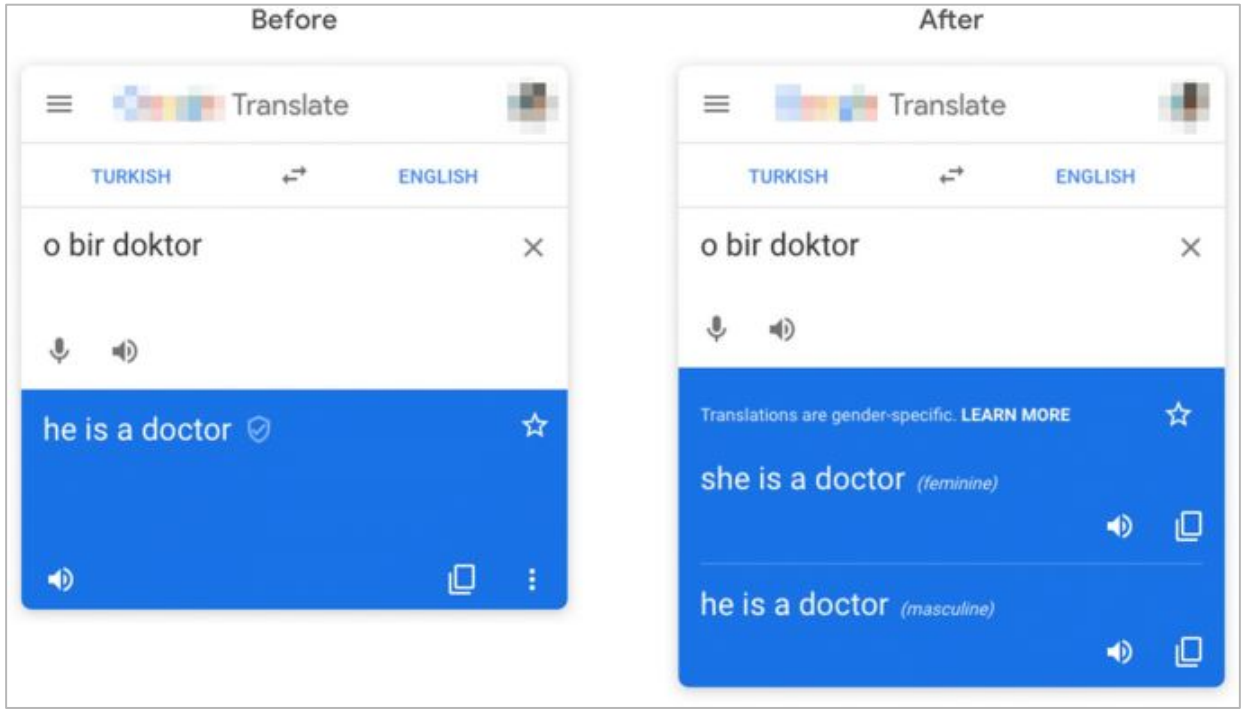


Image source: Marcelo O. R. Prates et al. Assessing gender bias in machine translation: a case study with Google Translate

The Ethics of AI Art

The Ethics of AI Art

Whereas NFTs incur a heavy environmental cost, the surging AI-driven art also raises serious questions about ownership, autonomy, and the future of art

Maya Garabedian / MutualArt Apr 11, 2023



Image source: https://www.mutualart.com/Article/The-Ethics-of-AI-Art/B1AAFE84D62BB4AC

Implications of artificial intelligence



https://youtu.be/gLoI9hAX9dw

Implications of artificial intelligence

infobae

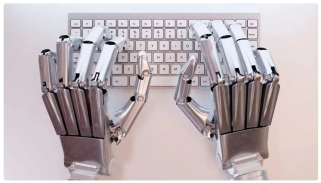
TECNO >

Inteligencia artificial puede adivinar contraseñas escuchando el sonido de las teclas

El software tiene una efectividad del 95% con grabaciones hechas con un celular

Por **Renato Silva**

08 Ago, 2023 01:30 p.m. EST



Inteligencia artificial puede adivinar contraseñas escuchando el sonido de las teclas con una precisión del 95%. (Popsci)

A Practical Deep Learning-Based Acoustic Side Channel Attack on Keyboards

Joshua Harrison[1], Ehsan Toreini[2], and Maryam Mehrnezhad[3]

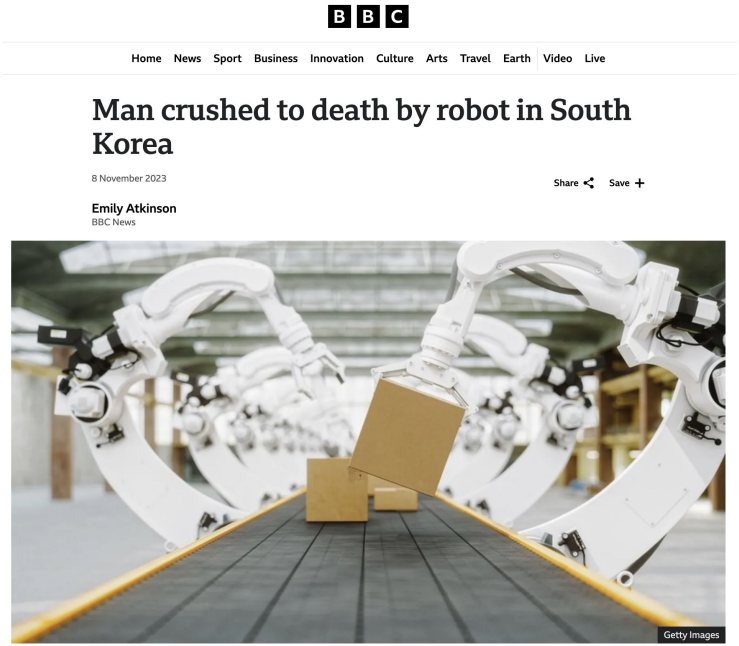
[1]Durham University, joshua.b.harrison@durham.ac.uk
[1]University of Surrey, e.toreini@surrey.ac.uk
[1]Royal Holloway University of London, maryam.mehrnezhad@rhul.ac.uk

August 3, 2023

Paper: https://arxiv.org/abs/2308.01074

Source:https://www.infobae.com/tecno/2023/08/08/inteligencia-artificial-puede-adivinar-contrasenas-escuchando-el-sonido-de-las-teclas/

Implications of artificial intelligence


BBC

Home News Sport Business Innovation Culture Arts Travel Earth Video Live

Man crushed to death by robot in South Korea

8 November 2023

Share Save

Emily Atkinson
BBC News

Getty Images



https://www.youtube.com/watch?v=2lfBkgha9v8

Source: https://www.bbc.com/news/world-asia-67354709

Summary

- Artificial intelligence
  - Definition
  - History
  - Use cases
  - Ethical issues

Week 2

IA débil y fuerte, machine learning, ciencia de datos, tipos de datos y métodos de recolección

08.09.2026 - 10.09.2026

Learning outcomes

- Describe the difference between the concepts of Weak AI and Strong AI.
- Describe the statistical data types.
- Describe the data types according to their structure.
- Describe the different methods to collect data.

The chinese room argument

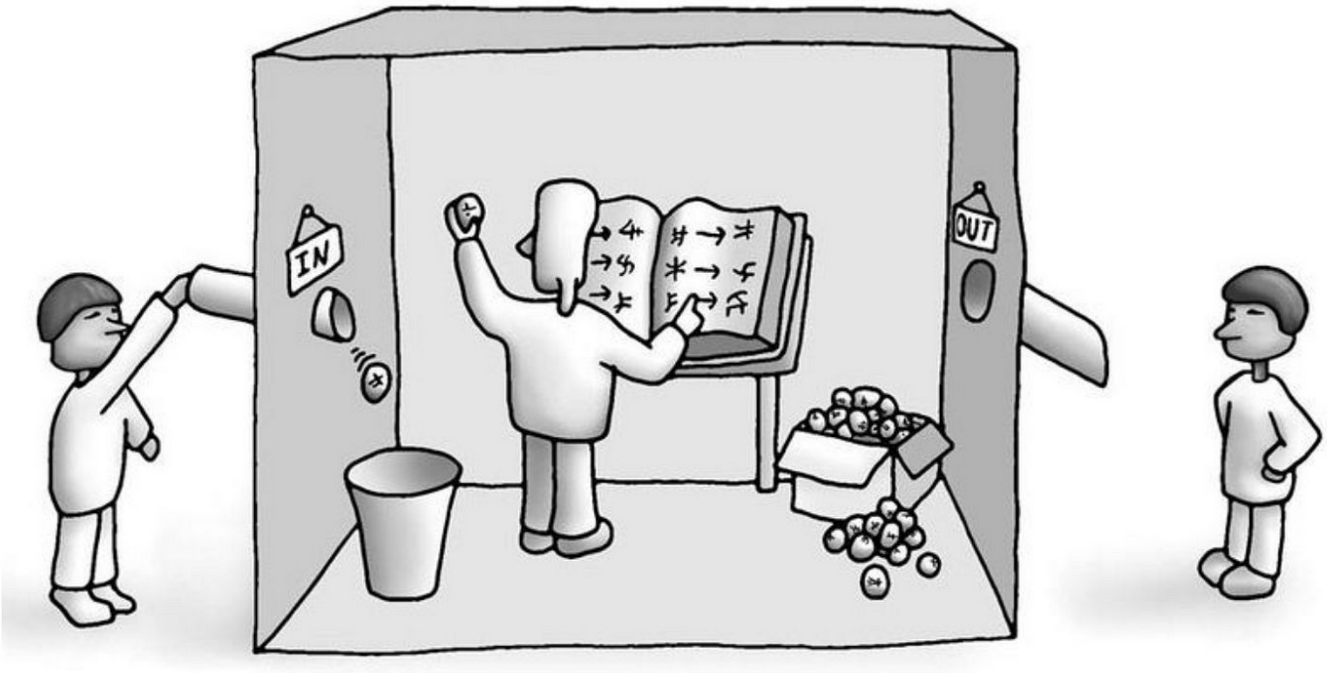


Image source: Wikicomms

The chinese room argument

- An experiment proposed by John Searle.
- The experiment consists of a person who does not know Chinese who is in a room while outside there is a Chinese person who sends him messages. The person in the room uses an instruction manual to be able to respond to what the Chinese person writes.

The chinese room argument

- The experiment gives the Chinese person the impression that he is talking to another person who speaks Chinese, but this does not mean that the person knows it.
- If a machine proves to be intelligent, for example by passing the Turing test, this does not mean that it is “intelligent” or behaves like humans as if it had a “mind”.

La habitación china, paso a paso

EJEMPLO

Entra por la ranura
你好吗?
(«¿Cómo estás?»)

→

Manual de instrucciones
«Si ves **你好吗?** copia y devuelve **我很好，谢谢**»
La persona solo compara la forma de los símbolos

→

Sale por la ranura
我很好，谢谢
(«Estoy bien, gracias»)

Desde afuera

Las respuestas son perfectas: parece que adentro hay alguien que habla chino.

Desde adentro

La persona no entiende ni una palabra: solo sigue reglas para manipular símbolos.

Analogía: un estudiante que copia respuestas de un solucionario puede entregar un deber perfecto sin entender el tema. Searle dice que un programa hace lo mismo: **procesar símbolos no es comprender**.

Weak AI

- Focus on performing specific tasks.
- Some examples of weak AI are: chatbots, spam filters in emails, Google Maps, smart assistants like Siri and Alexa.

Strong AI

- Ability to perform actions in the same way as a human being would do. It can solve problems and learn new skills on its own.
- Use of the learned knowledge in various tasks, and adaptation to the environment in case of unforeseen events.

IA débil vs. IA fuerte

EJEMPLO

	IA débil (estrecha)	IA fuerte (general)
Qué hace	Una tarea específica, muy bien.	Cualquier tarea intelectual que haría un humano.
Ejemplo	Siri pone una alarma; Deep Blue juega ajedrez; un filtro de spam clasifica correos.	Un asistente que aprende cocina, luego medicina y luego a manejar, solo y sin reprogramarlo.
Si cambia la tarea	Deep Blue no puede jugar damas; el filtro de spam no puede traducir.	Se adapta a situaciones imprevistas usando lo que ya sabe.
¿Existe hoy?	Sí Toda la IA real actual es IA débil.	No Es hipotética; aparece en la ciencia ficción (JARVIS, HAL 9000).

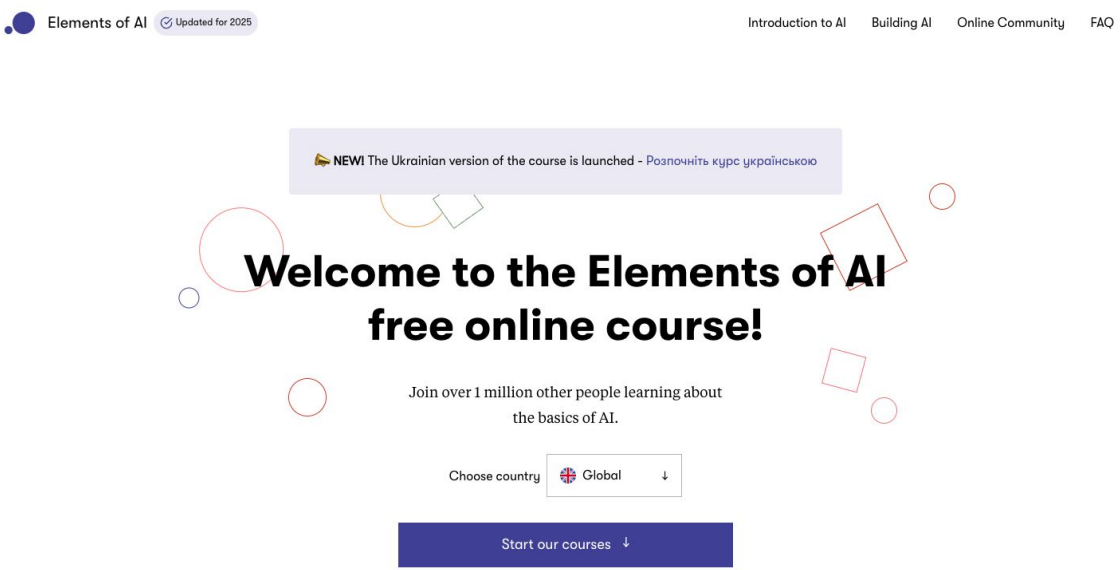
Truco para recordar: IA débil = **especialista** (un solo oficio); IA fuerte = **todólogo** que aprende cualquier oficio por su cuenta.

Philosophy of artificial intelligence

Discipline that seeks to understand the nature of artificial intelligence and its implications in the legal, social and ethical aspects. This philosophy seeks to answer the following questions:

- Can a machine act intelligently? Can it solve a problem that a person would solve by thinking?
- Are human intelligence and machine intelligence the same thing? Is the human brain essentially a computer?
- Can a machine have a mind, mental states and consciousness like humans? Can it sense how things are?

Additional content



https://www.elementsofai.com/

AI and the different disciplines

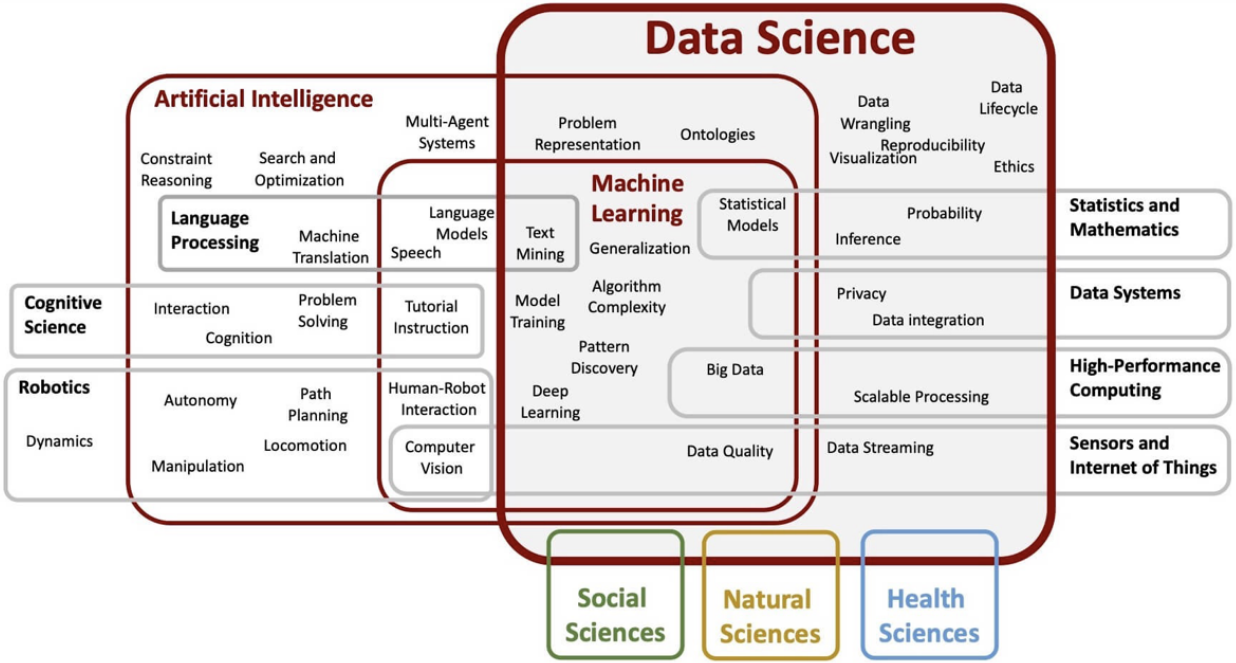
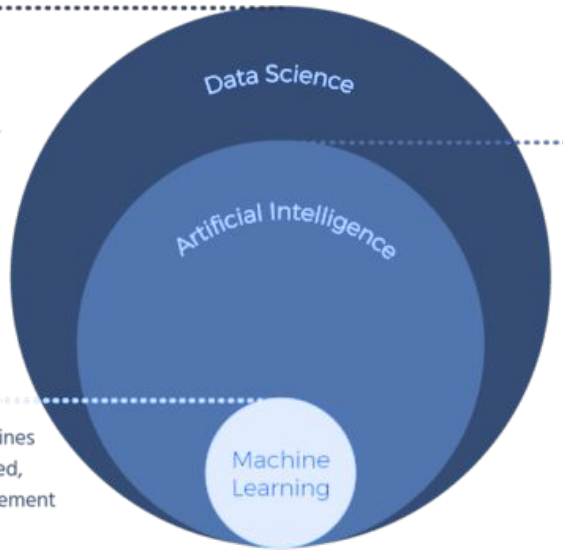


Image source: "Data Science as Discipline Diagram, Data Science Program, Viterbi School of Engineering, University of Southern California"

Data science vs machine learning vs AI

Data Science

- Collection, preparation, and analysis of data
- Leverages AI/ML, research, industry expertise, and statistics to make business decisions



Artificial Intelligence

- Technology for machines to understand/interpret, learn, and make 'intelligent' decisions
- Includes Machine Learning among many other fields

Machine Learning

- Algorithms that help machines improve through supervised, unsupervised, and reinforcement learning
- Subset of AI and Data Science tool

Image source: Braincube

Machine learning

- A subfield of artificial intelligence that focuses on statistics.
- The computer solves problems by learning from certain data and then using that experience to solve the problem in new situations.
- Systems improve the way they perform a specific task as they gain experience or accumulate data.

Programación tradicional vs. machine learning

EJEMPLO

Programación tradicional

Datos + **Reglas** → Respuestas

```
if color == "naranja" and textura == "rugosa":
    fruta = "naranja"
else:
    fruta = "manzana"
```

Tú escribes las reglas. ¿Y una manzana amarilla rugosa?

Machine learning

Datos + **Respuestas** → Reglas (modelo)

peso (g)	color	textura	fruta ✔
150	rojo	lisa	manzana
170	naranja	rugosa	naranja
140	amarillo	lisa	manzana
…	…	…	…

El algoritmo descubre las reglas a partir de 1 000 frutas ya etiquetadas.

Con **más ejemplos** (experiencia), el modelo mejora: eso es lo que significa que el sistema «aprende».

Data Science

- A multidisciplinary approach that combines several fields, such as machine learning, statistics, and computer science (algorithms, applications).
- A discipline that focuses on analyzing large volumes of data to extract information, understand patterns, and subsequently make decisions.

Ciencia de datos en una tienda

EJEMPLO

Pregunta del negocio: **«¿Por qué bajaron las ventas en agosto?»**

1. Recolectar

Ventas diarias, clima, promociones y visitas a la web de los últimos 2 años.

2. Limpiar

Quitar ventas duplicadas, corregir fechas mal escritas, completar datos faltantes.

3. Analizar

Gráficos y estadística: las ventas caen los días de lluvia y cuando no hay promoción.

4. Modelar (ML)

Un modelo predice las ventas de la próxima semana según el pronóstico del clima.

5. Decidir

Lanzar promociones online los días con pronóstico de lluvia y reforzar el delivery.

Combina **estadística** (analizar), **computación** (procesar muchos datos) y **machine learning** (predecir) para tomar **decisiones**.

Data vs information

- Data
  - From Latin *datum* <<something given>>.
  - Symbolic representation of a situation or scenario that lacks meaning.
- Information
  - From the Latin *informatio* <<to shape the mind, instruct, teach>>.
  - Processed data that is interpreted and leads to a message.

Information and context

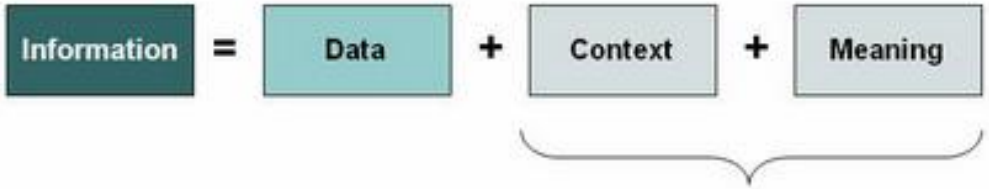
- Information is the result of data and context.
- Context refers to the unit of measurement in which the data is found.

Examples:

- 1.50 meters -> 1.50 (data) - meters (context).
- Black hair -> hair (context) - black (data).
- 6 dogs -> 6 (data) - dogs (context).

Information and context

INFORMATION = DATA + CONTEXT + MEANING



Data – raw facts and figures

Information – data that has been processed (in a context) to give it meaning

Image source: infotech100.weebly

Dato + contexto = información

EJEMPLO

Dato (sin significado)	Contexto	Información (mensaje)
`38.5`	Temperatura corporal de un paciente, en °C	Tiene fiebre: debería ver a un médico.
`593`	Código telefónico de país	+593 corresponde a Ecuador.
`Rojo`	Color actual del semáforo	Debes detenerte.
`65`	Nota sobre 100 en un curso que pide mínimo 70	No alcanza el mínimo para aprobar.

El mismo dato **«65»** cambia de significado según el contexto: 65 kg, 65 km/h, 65 años…

Un dato **no tiene que ser un número**: «Rojo» también es un dato (cualitativo).

Statistical data types

- Quantitative
  - Discrete
  - Continuous
- Qualitative
  - Ordinal
  - Nominal

Statistical data types

- Quantitative
    - Numerical values can be analyzed by obtaining statistics such as averages, and percentiles, among others.
- Qualitative
    - Attributes that can take a numerical value but lack mathematical meaning.

Quantitative data

- a.k.a *numerical*
- Quantitative data can be:
  - Discrete
    - Integer numbers, for instance, 15, 40
  - Continuous
    - Float numbers, for instance 3.15, 23.2

Qualitative data

- a.k.a *categorical*
- Qualitative data can be:
  - Nominal
    - Variables that cannot be ordered, por ejemplo: female, male.
  - Ordinal
    - Variables that can be ordered, for instance: low, medium, high.

Statistical data types - examples

- Quantitative
    - Weight, height, number of students in a classroom
- Qualitative
    - Gender, ethnicity, hometown

Clasifica las variables de una encuesta estudiantil

EJEMPLO

Variable	Valor de ejemplo	Tipo de dato
Edad (en años cumplidos)	20	Cuantitativa discreta
Número de materias inscritas	6	Cuantitativa discreta
Estatura	1.72 m	Cuantitativa continua
Promedio general	8.45	Cuantitativa continua
Ciudad de origen	Guayaquil	Cualitativa nominal
Número de celular	0991234567	Cualitativa nominal ¡aunque sea un número!
Nivel de inglés	A2 < B1 < B2	Cualitativa ordinal
Talla de camiseta	S < M < L	Cualitativa ordinal

Preguntas clave: ¿tiene sentido sacar el promedio? Si no → **cualitativa** (promediar celulares no significa nada). ¿Se puede ordenar? → **ordinal**. ¿Puede tener decimales? → **continua**.

Graphs according to the data type

- Nominal
  - Pie chart



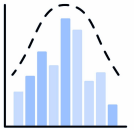
- Ordinal
  - Bar graph



- Discrete
  - Bar graph



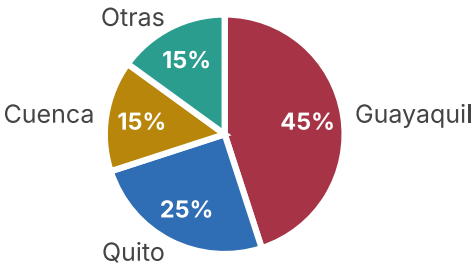
- Continuous
  - histogram



Un gráfico para cada tipo de dato

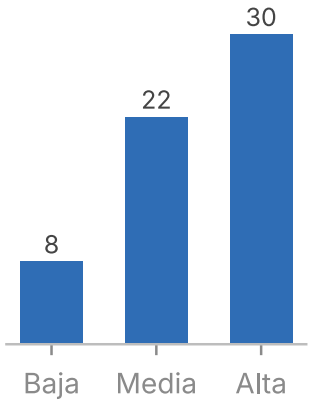
EJEMPLO

Nominal: ciudad



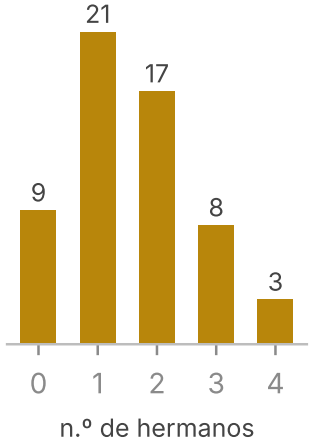
Pocas categorías sin orden → **pastel**
(partes de un todo)

Ordinal: satisfacción



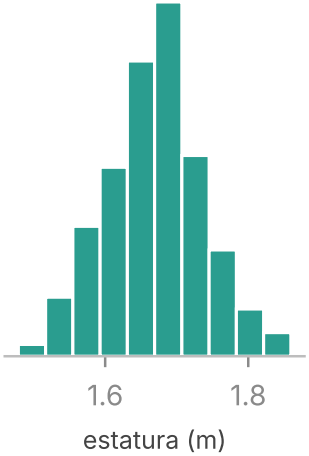
Categorías con orden → **barras**
respetando el orden

Discreta: hermanos



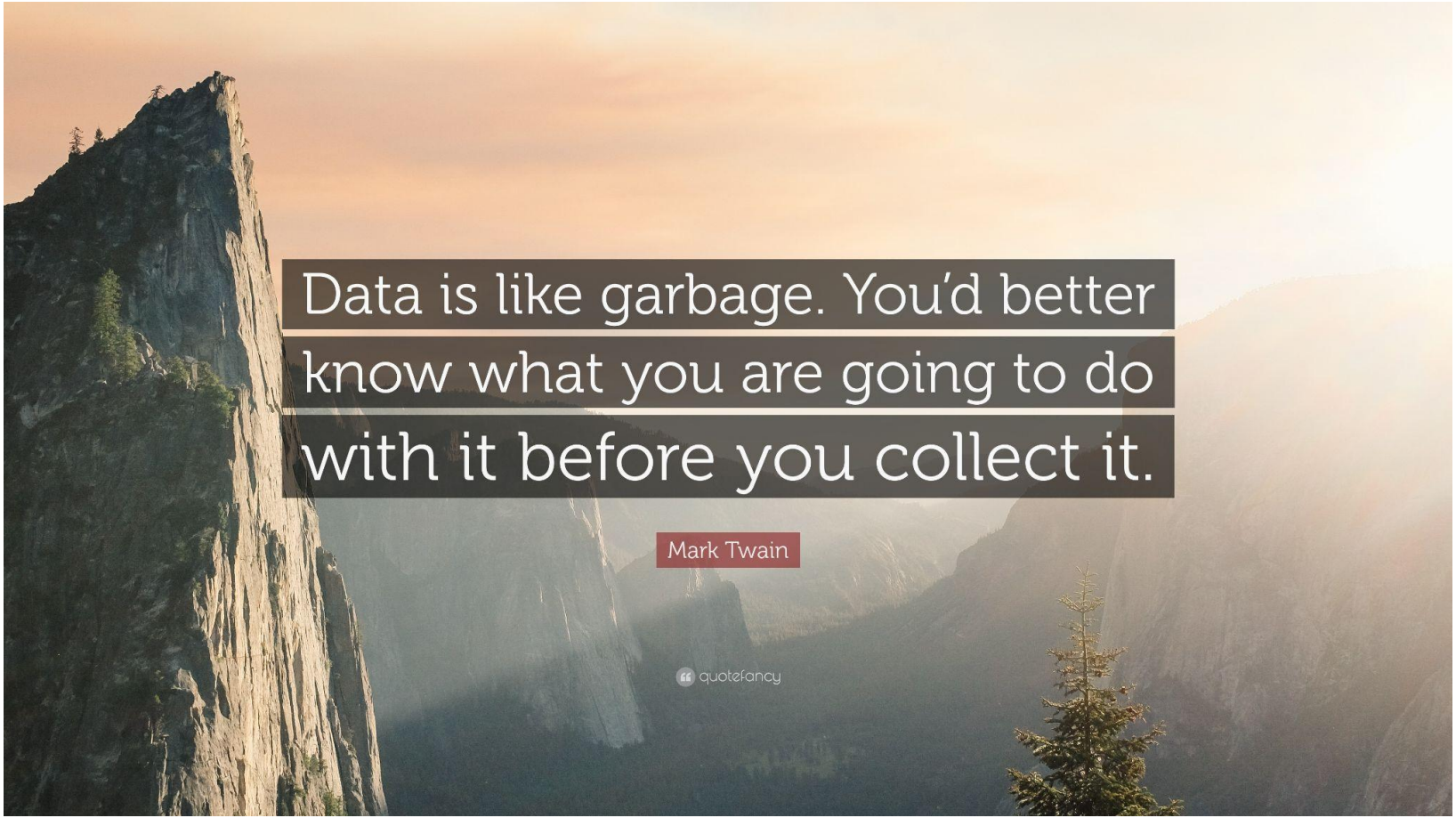
Conteos enteros → **barras** separadas

Continua: estatura



Valores con decimales → **histograma**
(barras pegadas por rangos)

Datos de ejemplo de una encuesta ficticia a 60–120 estudiantes.


Data is like garbage. You'd better know what you are going to do with it before you collect it.
Mark Twain
quotefancy

Data types according to the structure

- Structured data
- Unstructured data
- Semi-structured data



Unstructured data

The university has 5600 students.
John's ID is number 1, he is 18 years old and already holds a B.Sc. degree.
David's ID is number 2, he is 31 years old and holds a Ph.D. degree. Robert's ID is number 3, he is 51 years old and also holds the same degree as David, a Ph.D. degree.

Semi-structured data

```
<University>
  <Student ID="1">
    <Name>John</Name>
    <Age>18</Age>
    <Degree>B.Sc.</Degree>
  </Student>
  <Student ID="2">
    <Name>David</Name>
    <Age>31</Age>
    <Degree>Ph.D. </Degree>
  </Student>
  ....
</University>
```



Structured data

ID	Name	Age	Degree
1	John	18	B.Sc.
2	David	31	Ph.D.
3	Robert	51	Ph.D.
4	Rick	26	M.Sc.
5	Michael	19	B.Sc.

Image source: Kensho Blog, Jorge Cardoso - Developing Dynamic Packaging Applications Using Semantic Web-Based Integration

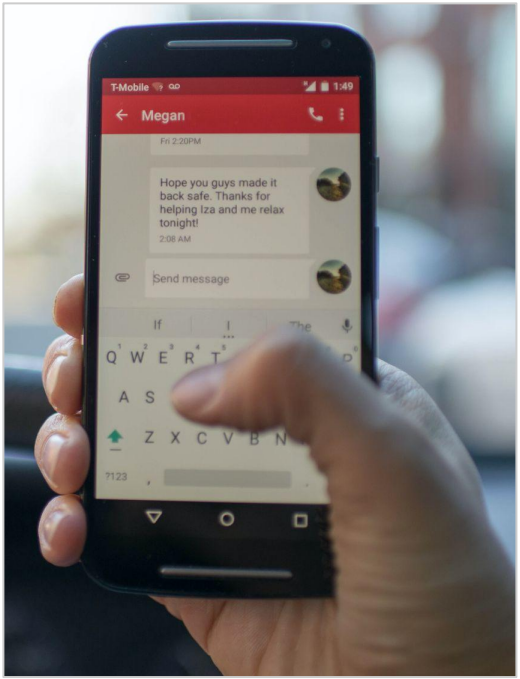
Structured data

- Sorted data can be obtained from different sources such as surveys, forms, excel sheets, databases, etc.
- Easy to handle given their structure (data organized in rows and columns).



Unstructured data

- Data lack a structure. It can be found in different sources such as social networks, videos, comments, call transcripts, etc.
- It requires more effort to handle given the complexity of the format.



Semi-structured data

- Data presents a certain structure since it employs labels to organize data. As a result, it eases the creation of hierarchies and groups. For instance, emails, JSON files, XML files, etc.
- This type of data is more easy to store than the unstructured data.



Image source: TapClicks

La misma información, tres estructuras

EJEMPLO

Estructurada (tabla)

nombre	edad	carrera
María	20	Sistemas
José	22	Derecho

Filas y columnas fijas: lista para Excel o una base de datos.

Semiestructurada (JSON)

```
{"nombre": "María",
 "edad": 20,
 "carrera": "Sistemas",
 "redes": ["IG", "TikTok"]}
```

Etiquetas (claves) organizan los datos, pero cada registro puede tener campos distintos.

No estructurada (texto)

«Holaa! soy María 😊 tengo 20 y estudio Sistemas en la UEES, me encanta TikTok»

Mensaje de WhatsApp: hay que procesarlo (p. ej., con IA) para extraer nombre, edad o carrera.

Más estructura = **más fácil de analizar**. Otros no estructurados: audios, videos, fotos, comentarios en redes.

Data according to the collection method

- Observational
  - Data is collected by observing a subject in a natural setting without altering the environment or any variable. For instance, the preferences of the people about a product or service, the behavior of the people in an specific place.
- Experimental
  - Data collected under controlled conditions. For instance, the A/B test to evaluate a webpage design.

Observacional vs. experimental

EJEMPLO

Observacional: solo miras

En la cafetería se anota cuántos cafés se venden por hora durante una semana, **sin cambiar nada**.

Hora	Cafés vendidos
07:00	48
10:00	21
13:00	35

Conclusión: la mayor demanda es temprano. No sabemos **por qué**.

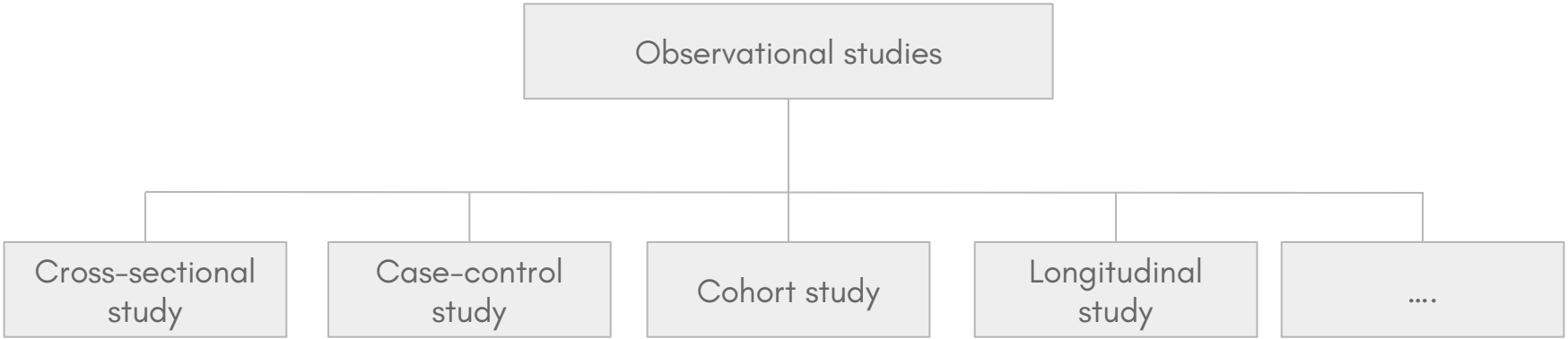
Experimental: controlas una variable (A/B test)

Una tienda online muestra a la mitad de visitantes un botón **gris** (A) y a la otra mitad uno **rojo** (B). Todo lo demás es igual.

Versión	Visitas	Compras	Tasa
A · gris	1 000	41	4.1 %
B · rojo	1 000	53	5.3 %

Como solo cambió el color, la diferencia se puede **atribuir** al botón.

Some types of observational studies



Cross-sectional study

Studies to observe different groups in a specific period of time. For instance, the predominance of an illness in a population.

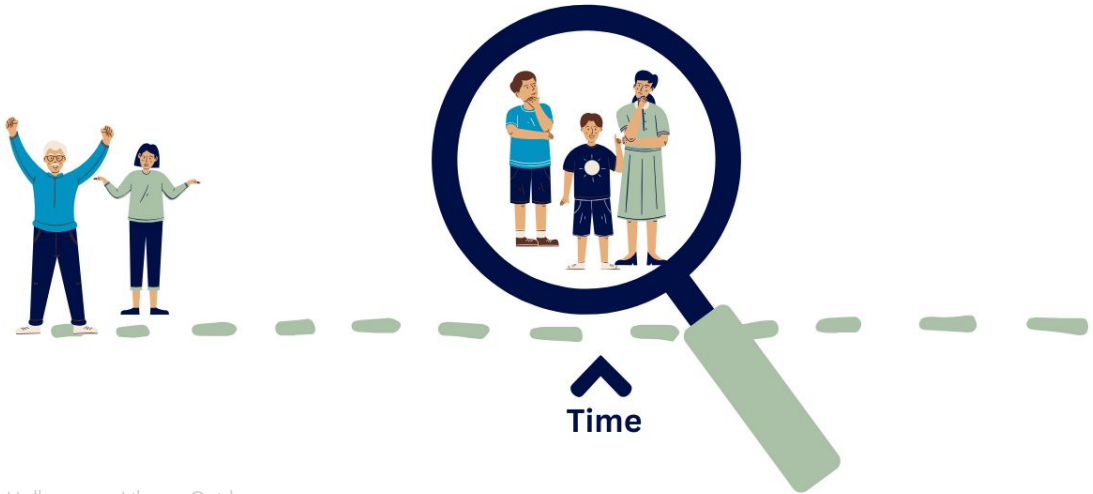


Image adapted from The University of Melbourne - Library Guides

Case-control study

Studies to investigate the causes of a disease or a condition by comparing two groups: one with the disease (case) with another group without it (control). The idea is to identify factors that could contribute to the disease. For instance, the link between high-fat diet and colon cancer by observing people with cancer (cases) and without it (controls) by examining their dietary habits.

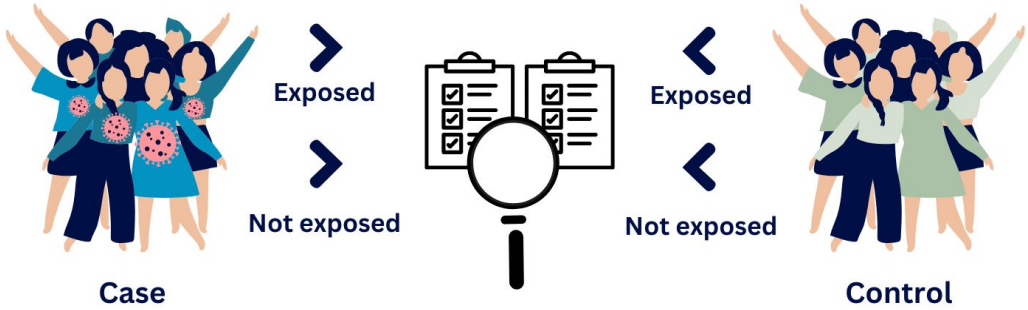


Image adapted from The University of Melbourne - Library Guides

Cohort study

Studies to observe a group of people (the cohort) over time. Usually, the cohort is split into groups that share certain characteristics or exposures, such as behaviours and environmental factors. The groups are tracked and eventually compared. For instance, analysis of the long-term effects of diet, lifestyle, and medication use on women's health.

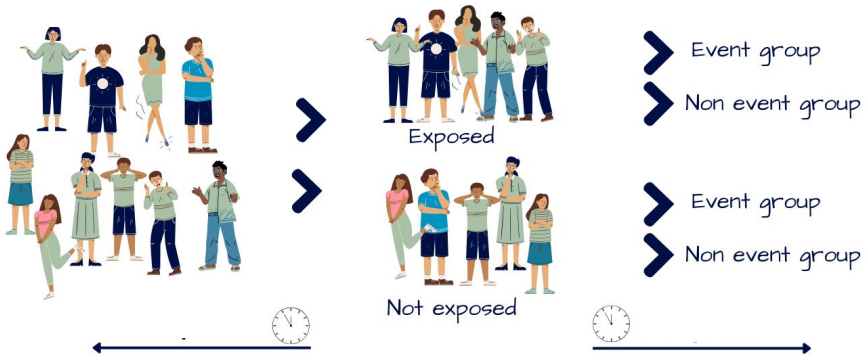


Image adapted from The University of Melbourne - Library Guides

Longitudinal study

Studies to observe subjects (same group) for a longer time (e.g., years). It is useful to find changes over time and identify trends. For instance, studying the lifestyles of individuals of similar characteristics uncovers factors that contribute to successful ageing.

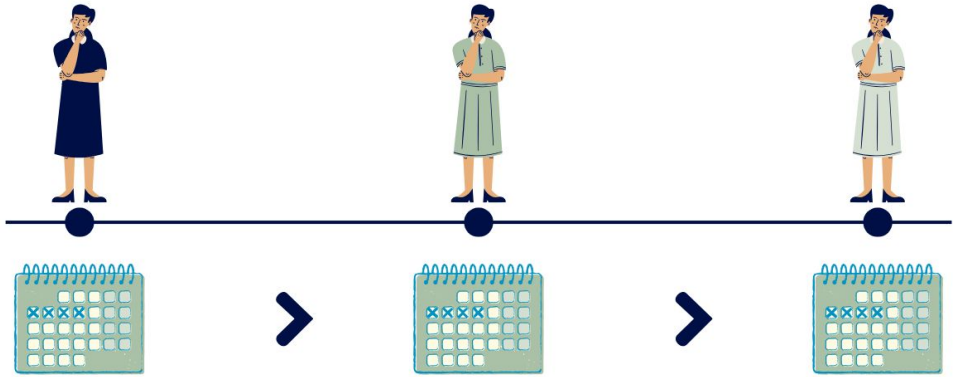


Image adapted from The University of Melbourne - Library Guides

Una pregunta, cuatro diseños de estudio

EJEMPLO

Pregunta: **¿El uso de redes sociales afecta el sueño de los universitarios?**

Transversal (una «foto»)

En octubre de 2026 se encuesta a 500 estudiantes, una sola vez, sobre horas en redes y horas de sueño.

Casos y controles (mira hacia atrás)

Se toman 50 estudiantes **con** insomnio (casos) y 50 **sin** insomnio (controles) y se compara su uso de redes en el último año.

Cohorte (grupos expuestos vs. no)

Se sigue a estudiantes de primer año que usan redes más de 4 h/día y a otros que usan menos de 1 h/día; tras 3 años se compara quiénes desarrollaron insomnio.

Longitudinal (el mismo grupo, años)

Se mide a los mismos 200 estudiantes cada año durante 5 años para ver cómo cambian sus hábitos de sueño y de redes.

Ninguno interviene: todos son **observacionales**. Cambia **cuándo** y **a quién** se observa.

Different sources for searching datasets

- Kaggle
- Google Dataset Search
- Data.Gov
- Datahub.io
- UCI Machine Learning Repository











Summary

- Definition of Machine learning and Data Science
- Statistical data types
- Data types according to the structure
- Data collection methods

Week 3

Jupyter Notebook, Google Colab y fundamentos de Python

15.09.2026 - 17.09.2026

Learning outcomes

- Describe the concept of Jupyter Notebook
- Describe the different types of cells available on Google Collab
- Create a notebook on Google Collab with Python

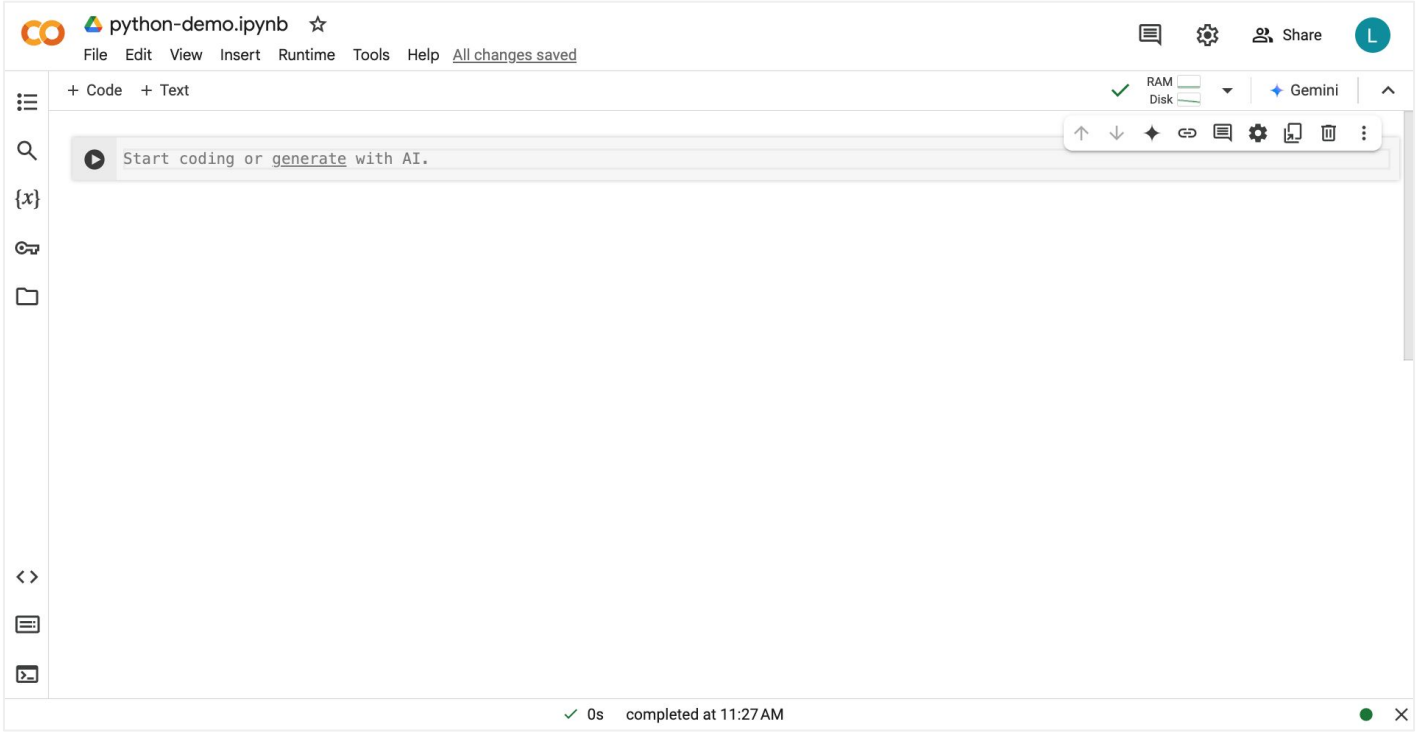
Jupyter Notebook

- A client-server application that facilitates editing and running notebooks (code) through a web browser.
- The execution of the code is possible using a kernel, which functions as a computational engine to perform the operations. There is a kernel for various languages; in the case of Python there is IPython.

Google Colaboratory

- a.k.a Colab or Google Colab.
- A tool created by Google in 2017 for the development of notebooks.
- A hosted Jupyter Notebook service that does not require any prior configuration for use.

A view of a notebook in Colab

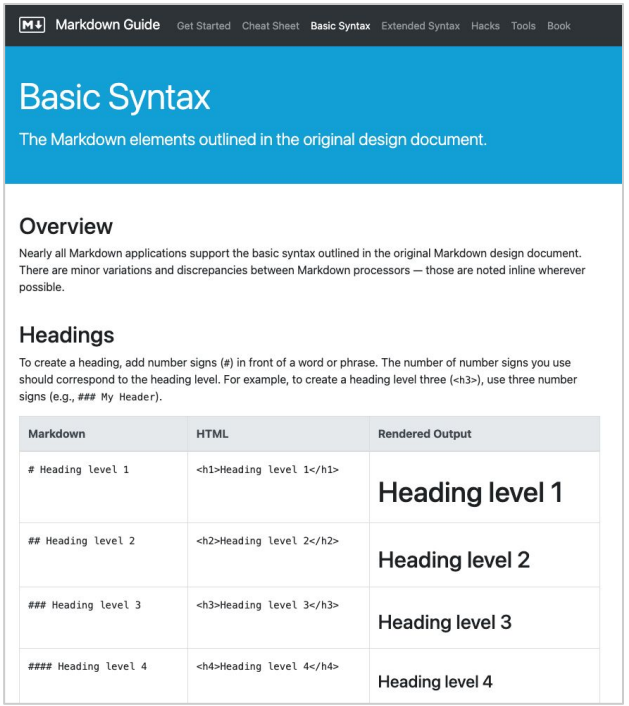


Types of cells in Colab

- Code cell
- Section header cell
- Text cell
- Scratch code cell

Markdown code

- Created by John Gruber in 2004.
- A lightweight markup language used to add formatting elements, links, and images to plain text files.



Markdown Guide Get Started Cheat Sheet **Basic Syntax** Extended Syntax Hacks Tools Book

Basic Syntax

The Markdown elements outlined in the original design document.

Overview

Nearly all Markdown applications support the basic syntax outlined in the original Markdown design document. There are minor variations and discrepancies between Markdown processors — those are noted inline wherever possible.

Headings

To create a heading, add number signs (#) in front of a word or phrase. The number of number signs you use should correspond to the heading level. For example, to create a heading level three (<h3>), use three number signs (e.g., ### My Header).

Markdown	HTML	Rendered Output
# Heading level 1	Heading level 1	Heading level 1
## Heading level 2	Heading level 2	Heading level 2
### Heading level 3	Heading level 3	Heading level 3
#### Heading level 4	Heading level 4	Heading level 4

Source - https://www.markdownguide.org/basic-syntax/

Markdown: lo que escribes vs. lo que ves

EJEMPLO

Lo que escribes en la celda de texto

```
# Mi primer notebook
## Objetivo
Analizar las **notas** del curso
con *Python*.

- Paso 1: cargar datos
- Paso 2: graficar

Ver [Colab](https://colab.google)
Usa `print()` para mostrar.
```

Lo que se muestra al ejecutarla

Mi primer notebook

Objetivo

Analizar las **notas** del curso con *Python*.

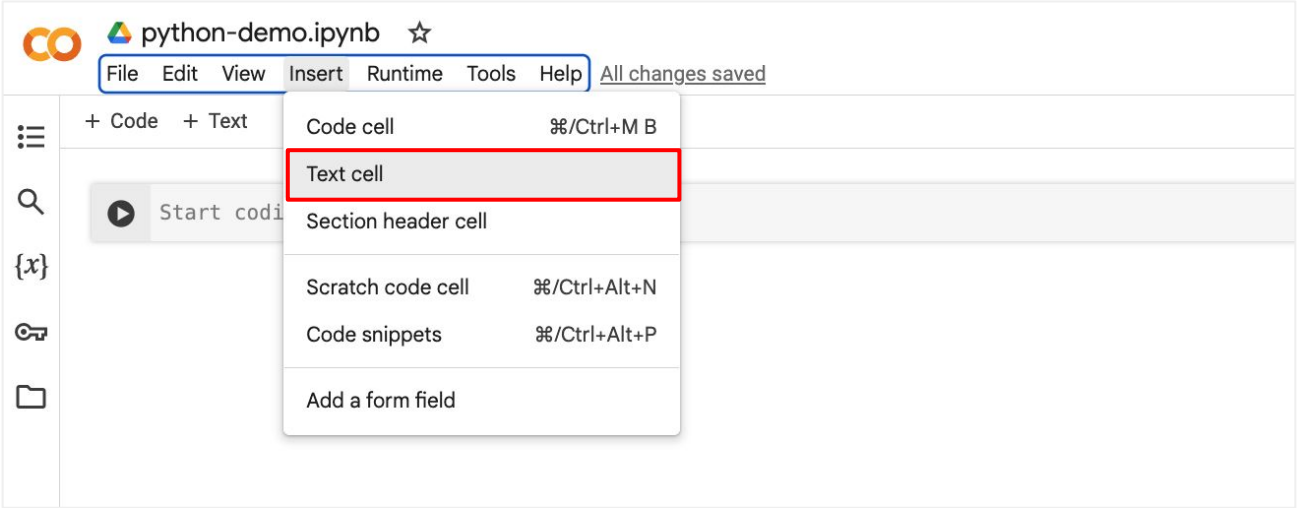
- Paso 1: cargar datos
- Paso 2: graficar

Ver Colab

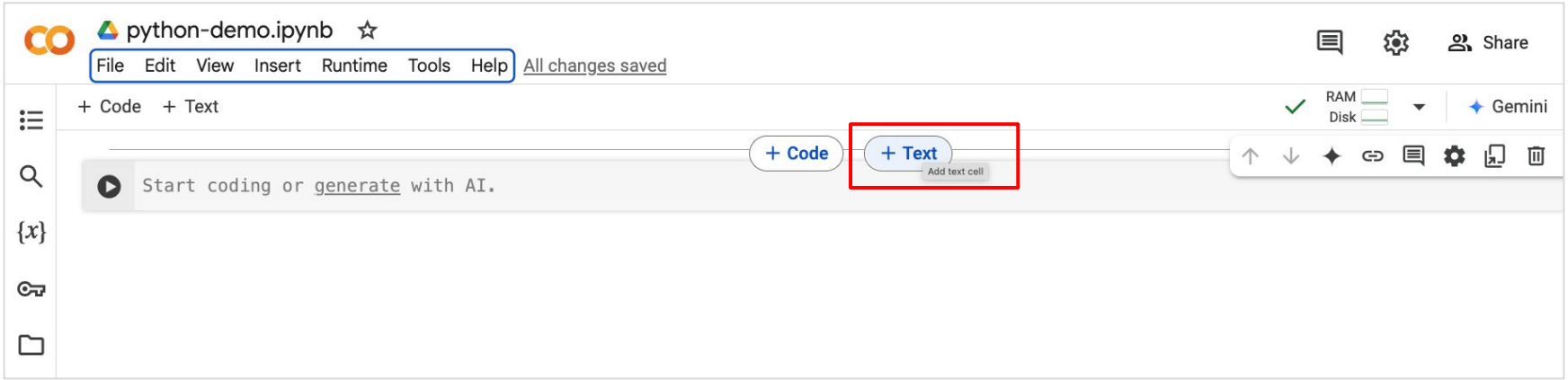
Usa `print()` para mostrar.

> # = título · ## = subtítulo · **negrita** · *cursiva* · - lista · [texto](url) enlace · `código`. Nota: un salto de línea simple no separa párrafos; deja una línea en blanco.

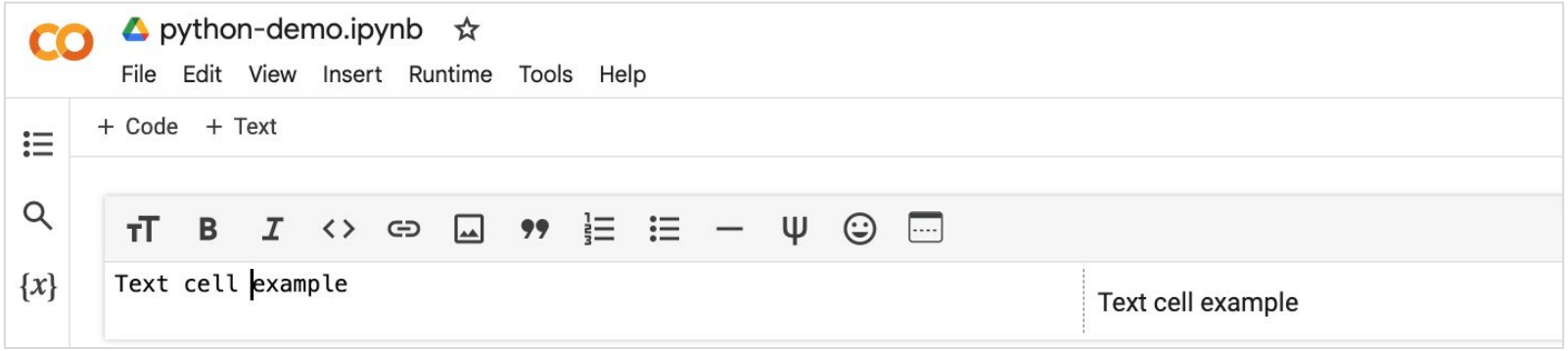
Inserting a text cell in Colab - option 1



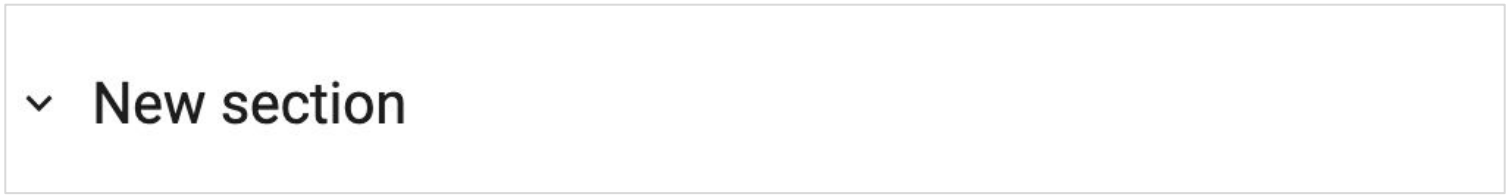
Inserting a text cell in Colab - option 2



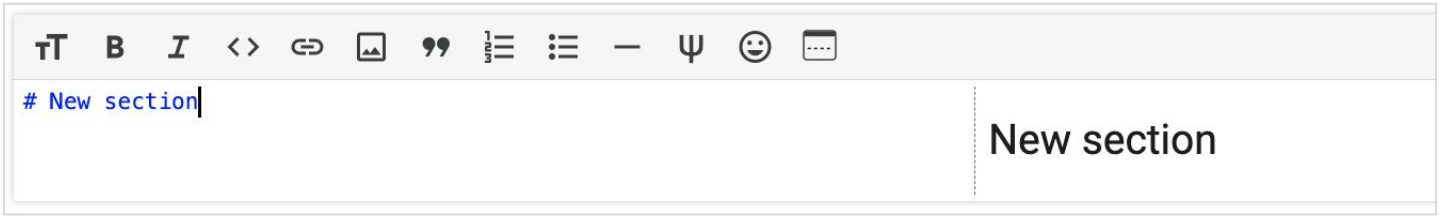
Text cell in Colab - example



Section header cell

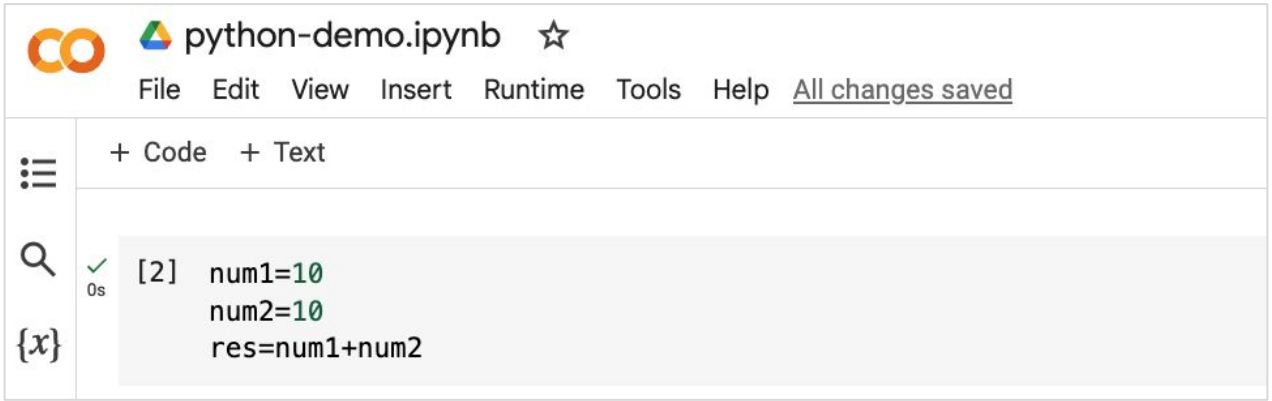


Double click in the cell to edit the text

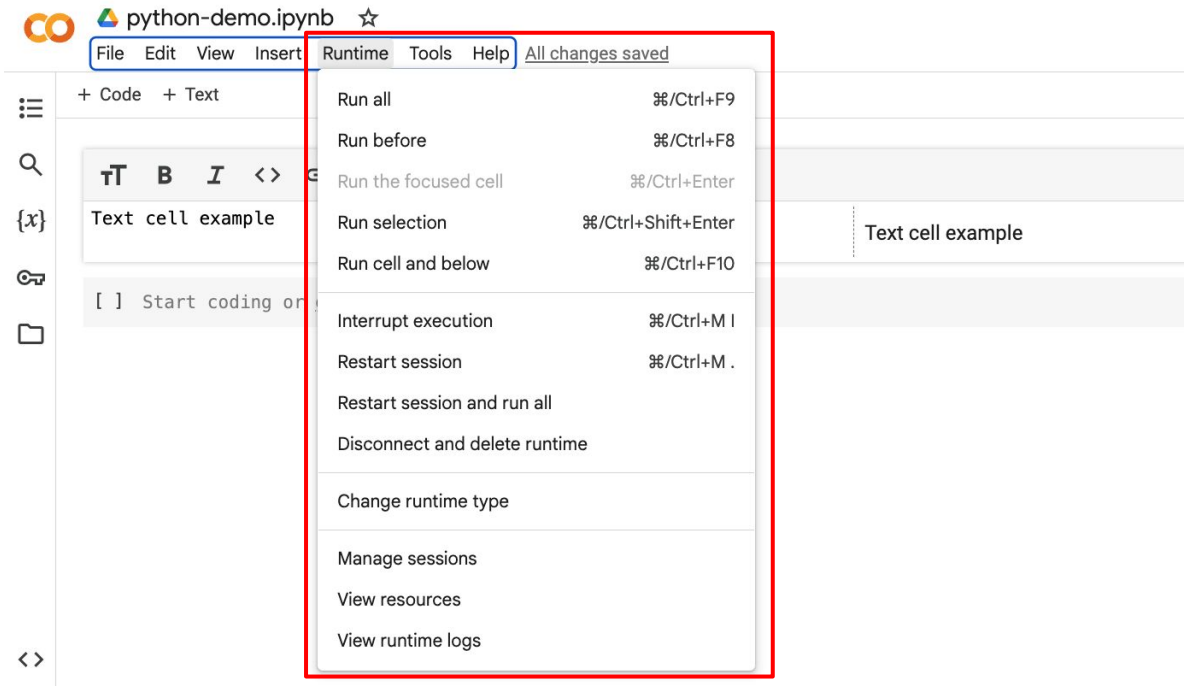


Code cell

The code cell shows a pair of brackets [ ] on the left, and when executed it shows a numeric value that indicates the order of execution of the cell with respect to the others.



Execution of cell in Colab



El número [n] indica el orden de ejecución

EJEMPLO

Ejecutas las celdas de arriba hacia abajo:

```
[1] x = 10
```

```
[2] x = x + 5
```

```
[3] print(x)
    15
```

Luego vuelves a ejecutar la 2.ª y la 3.ª celda:

```
[1] x = 10
```

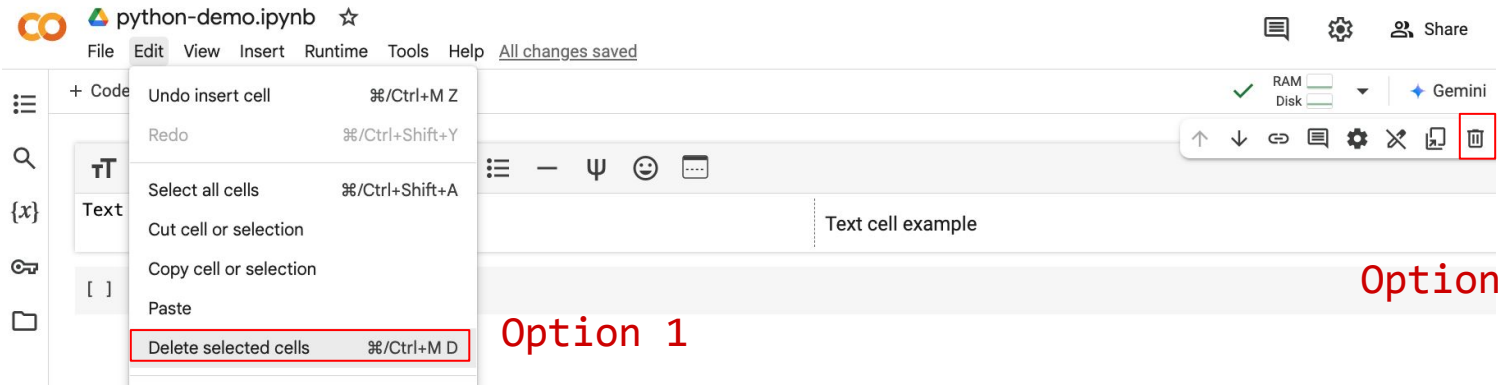
```
[4] x = x + 5
```

```
[5] print(x)
    20
```

El notebook **recuerda** las variables entre celdas. Lo que importa es el **orden en que ejecutas**, no el orden en que aparecen.

Si los resultados no tienen sentido: **Runtime → Restart session and run all** para ejecutar todo desde cero.

Deleting a cell in Colab



Python

- Created by Guido van Rossum in the late 80s.
- High-level, interpreted, cross-platform programming language.
- Code blocks are defined using indentation and do not require braces {}.
- Current version: 3.14.7 (August 2026).



Hello world in Python

Use of the function print(), which receives strings accompanied by quotation marks (single or double).

```
[1]  print("Hello world")

     Hello world
```

Example - sum of two numbers in Python

```
[ ]  num1 = 10
     num2 = 20
     res = num1 + num2
```

```
[ ]  print(res)
```

```
     30
```

Different formats to show values using the print() function

Using print with comma

```
[3]  print("The sum of num1 and num2 is: ", res)
```

```
The sum of num1 and num2 is:  20
```

Using print with f-string

```
[4]  print(f"The sum of num1 and num2 is: {res}")
```

```
The sum of num1 and num2 is: 20
```

Using print with + and str()

```
print("The sum of num1 and num2 is "+ str(res))
```

```
The sum of num1 and num2 is 20
```

print(): cuatro formas de mostrar valores

EJEMPLO

```
[1] nombre = "Ana"
    edad = 20
    promedio = 8.756
```

```
[2] print("Nombre:", nombre, "- Edad:", edad)
```

```
Nombre: Ana - Edad: 20
```

Con comas: separa automáticamente con espacios.

```
[3] print(f"{nombre} tiene {edad} años")
```

```
Ana tiene 20 años
```

f-string: las variables van entre llaves { }.

```
[4] print(f"Promedio: {promedio:.2f}")
```

```
Promedio: 8.76
```

:.2f redondea a 2 decimales al mostrar.

```
[5] print("Edad: " + str(edad))
```

```
Edad: 20
```

Con +: todo debe ser texto, por eso se usa str().

```
[6] print("Edad: " + edad)
```

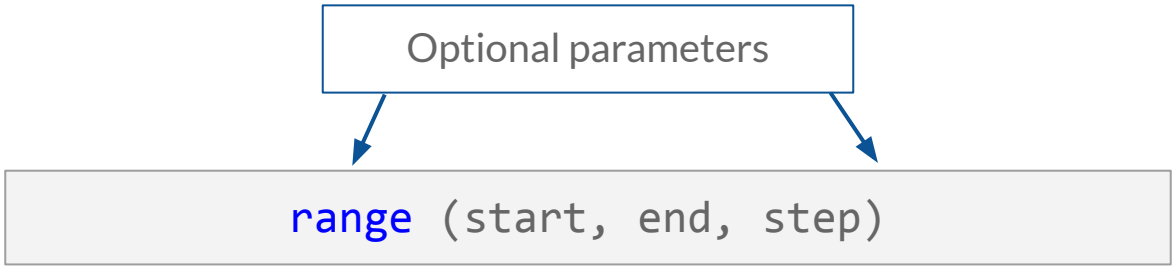
```
TypeError: can only concatenate str (not "int") to str
```

Error típico: no se puede unir texto (str) con un número (int).

The range function

- A built-in function to generate a sequence of numbers.
- By default, the numbers start from 0 up to the number indicated with steps of 1.

Sintax:



range(start, end, step) con ejemplos

EJEMPLO

Código	Resultado
`list(range(5))`	`[0, 1, 2, 3, 4]`
`list(range(2, 8))`	`[2, 3, 4, 5, 6, 7]`
`list(range(0, 10, 3))`	`[0, 3, 6, 9]`
`list(range(10, 0, -2))`	`[10, 8, 6, 4, 2]`
`list(range(1, 6, 2))`	`[1, 3, 5]`

Regla de oro

El **end nunca se incluye**: `range(2, 8)` llega hasta 7, no hasta 8.

Valores por defecto

`start = 0` y `step = 1`. Con un solo número, ese número es el **end**.

Paso negativo

Cuenta hacia atrás: `range(10, 0, -2)` → 10, 8, 6, 4, 2.

Se usa `list()` solo para ver los números; en un `for` no hace falta.

Using for loop in Python

```
for num in range(0,13,2):
  print(num)
```

```
0
2
4
6
8
10
12
```

Bucle for: recorrer una lista y usar range()

EJEMPLO

Ejemplo 1 · Promedio de notas

```
[1] notas = [85, 92, 78]
    total = 0
    for n in notas:
        total = total + n
        print("Sumé", n, "-> total =", total)
    print("Promedio:", total / len(notas))
```

```
Sumé 85 → total = 85
Sumé 92 → total = 177
Sumé 78 → total = 255
Promedio: 85.0
```

Ejemplo 2 · Tabla del 7 con range()

```
[2] for i in range(1, 5):
        print(f"7 x {i} = {7 * i}")
```

```
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
```

Traza del ejemplo 1 (qué pasa en cada vuelta)

Vuelta	n	total
inicio	—	0
1	85	0 + 85 = 85
2	92	85 + 92 = 177
3	78	177 + 78 = 255

El código **con sangría** (indentación) se repite en cada vuelta; el `print("Promedio…")` sin sangría se ejecuta una sola vez, al final: 255 / 3 = 85.

If-elif-else statements

```
num = 2

if num <0:
  print("The number is negative")
elif num > 10:
  print("The number is greater than 10")
else:
  print("The number is greater than 0 and less than 10")
```

```
The number is greater than 0 and less than 10
```

if-elif-else: calificar una nota

EJEMPLO

```
[1] nota = 72

    if nota >= 90:
        print("Excelente")
    elif nota >= 70:
        print("Aprobado")
    else:
        print("Reprobado")
```

```
Aprobado
```

nota	¿⩾ 90?	¿⩾ 70?	Imprime
95	✔	(no se revisa)	**Excelente**
72	✘	✔	**Aprobado**
70	✘	✔ (incluye el 70)	**Aprobado**
40	✘	✘	**Reprobado**

Se revisan las condiciones **en orden** y solo se ejecuta el **primer** bloque verdadero. 95 también es ⩾ 70, pero ya entró en «Excelente».

`else` no lleva condición: atrapa todo lo que no cumplió las anteriores.

Functions in Python

```
[10] def showMessage():
       print("Invoking the method showMessage")
```

```
[11] showMessage()
```

```
Invoking the method showMessage
```

Functions in Python

```
[23] def calculateOperation(num1,num2,op="+"): # a function with a default parameters
       if op == "-":
         res = num1-num2
       elif op == "*":
         res = num1*num2
       elif op == "/":
         res = num1/num2
       else:
         res = num1+ num2
       print("The result is: ", res)
```

Functions in Python

```
[24] calculateOperation(3,2) # invoke a function without specifying the operator

     The result is:  5

[25] calculateOperation(3,2,"*") # invoke a function specifying the * operator

     The result is:  6
```

Funciones: parámetros, valor por defecto y return

EJEMPLO

```
[1] def calcular_area(base, altura=1):
        return base * altura
```

def define la función; se ejecuta solo cuando la llamas.

```
[2] calcular_area(5, 3)
```

```
15
```

Se envían los dos parámetros.

```
[3] calcular_area(5)
```

```
5
```

altura no se envía: usa el valor por defecto (1).

```
[4] resultado = calcular_area(4, 2) + 1
    resultado
```

```
9
```

return devuelve el valor y se puede seguir usando.

```
[5] def saludar(nombre):
        print("Hola", nombre)

    x = saludar("Ana")
    print(x)
```

```
Hola Ana
None
```

return vs. print: `print` solo muestra en pantalla. Por eso `saludar()` imprime «Hola Ana» pero devuelve **None**: no tiene return.

Collections in Python

Some basic collections in Python are:

- List
- Tuple
- Dictionary

Lists in Python

- Data type used to store multiple values in a variable.
- A list is ordered, modifiable, and supports repeated values.
- Use of indexes to access elements starting from 0.

Sintax:

```
variable = [elem1, elem2, elem3]
```

* Another form to create a list is using the function list().

Lists in Python

```
[12] lst = [1,3,5,7] # Create a list
```

Obtain the number of elements in a list

```
[13] len(lst)
```

```
4
```

Listas: índices, modificar y agregar

EJEMPLO

```
[1] frutas = ["mango", "guineo", "piña"]
    print(frutas[0], frutas[-1])
```

```
mango piña
```

```
[2] frutas[1] = "maracuyá"   # se puede modificar
    frutas.append("mango")   # admite repetidos
    frutas
```

```
['mango', 'maracuyá', 'piña', 'mango']
```

Ordenada, modificable y con valores repetidos.

```
[3] len(frutas)
```

```
4
```

```
[4] frutas[4]
```

```
IndexError: list index out of range
```

Índices de la lista final

índice	0	1	2	3
valor	mango	maracuyá	piña	mango
índice negativo	-4	-3	-2	-1

Se cuenta **desde 0**. Con 4 elementos, el último índice es 3; por eso `frutas[4]` da **IndexError**.

`-1` siempre es el último elemento, sin importar el tamaño.

Tuples

- A data structure that stores data that is immutable, meaning it cannot be modified once created.
- A tuple can store elements of different types and compound elements such as lists, tuples, etc.
- Tuples are created using parentheses () and separating elements with commas.

```
tuple = (element1, element2, … )
```

Tuples in Python

```
[21] tpl = (2,4,5,6)
     tpl
```

```
(2, 4, 5, 6)
```

```
[22] type(tpl) # The type function is used to verify the data type of a variable
```

```
tuple
```

Tuplas: como una lista que no se puede modificar

EJEMPLO

```
[1] punto = (3, 4)
    punto[0]
```

```
3
```

```
[2] punto[0] = 10
```

```
TypeError: 'tuple' object does not support item assignment
```

Inmutable: una vez creada no se puede cambiar.

```
[3] estudiante = ("Ana", 20, [85, 92])   # tipos mezclados
    type(estudiante)
```

```
<class 'tuple'>
```

	Lista	Tupla
Se escribe con	`[ ]`	`( )`
¿Modificable?	Sí	No
Ejemplo de uso	Lista de compras, notas que cambian	Coordenadas, días de la semana, fecha (día, mes, año)

> Úsala cuando los datos **no deben cambiar** por error: `dias = ("lun", "mar", "mié")`.

Slicing in Python

An operator to extract or access portions of sequences like strings, lists, and tuples

Syntax:

```
[start:stop:end]
```

- `start`. (Optional). Starting index of the slice. Defaults value is 0.
- `stop`. (Optional). The last index of the slice. Defaults value is len(sequence).
- `step`. (Optional). Step value of the slice. Defaults value is 1.

Slicing in Python - example

```
lst = [10,8,6,4,2]
lst2 = lst[:] # a copy of lst
lst2[1] = 10
print(lst2[3:])
print(lst2[:3])
print(lst2[1:-1])
print(lst2[-1:1])
print(lst2)
```

Slicing: resultados del ejemplo de la clase

EJEMPLO

```
[1] lst = [10,8,6,4,2]
    lst2 = lst[:] # a copy of lst
    lst2[1] = 10
    print(lst2[3:])
    print(lst2[:3])
    print(lst2[1:-1])
    print(lst2[-1:1])
    print(lst2)
    print(lst)
```

```
[4, 2]
[10, 10, 6]
[10, 6, 4]
[]
[10, 10, 6, 4, 2]
[10, 8, 6, 4, 2]
```

lst2 después de `lst2[1] = 10`

índice	0	1	2	3	4
valor	10	**10**	6	4	2
negativo	-5	-4	-3	-2	-1

- `[3:]` del índice 3 al final → **[4, 2]**
- `[:3]` del inicio hasta el 2 (el 3 no entra) → **[10, 10, 6]**
- `[1:-1]` del 1 hasta antes del último → **[10, 6, 4]**
- `[-1:1]` empieza en el último y avanza hacia la derecha: nunca llega al 1 → **[ ]**
- `lst` no cambió porque `lst[:]` creó una **copia**.

Slicing con texto: [inicio:fin:paso]

EJEMPLO

```
[1] palabra = "PYTHON"
    print(palabra[0:2])
    print(palabra[2:])
    print(palabra[-3:])
    print(palabra[::2])
    print(palabra[::-1])
```

```
PY
THON
HON
PTO
NOHTYP
```

	P	Y	T	H	O	N
índice	0	1	2	3	4	5
negativo	-6	-5	-4	-3	-2	-1

Slice	Se lee como...	Resultado
`[0:2]`	índices 0 y 1	PY
`[2:]`	del 2 hasta el final	THON
`[-3:]`	los últimos 3	HON
`[::2]`	todo, saltando de 2 en 2 (0, 2, 4)	PTO
`[::-1]`	todo, al revés	NOHTYP

El tercer valor es el **paso** (step): cuántas posiciones avanzar. Funciona igual en listas, tuplas y textos.

Dictionaries

- A data structures that store data as key pairs
- A dictionary can store numbers, text strings, lists, and other dictionaries
- The elements in a dictionary consist of a key and a value

```
dictionary = {key1:value1,.....}
```

Dictionaries

- Dictionaries are mutable
- Dictionaries have no order
- Keys are unique to a dictionary

Dictionaries in Python

```
[14] dic = {} # Empty dictionary

[15] print(dic)

     {}

[16] dic1={'user1':'abohr', 'id':100}
     dic2={'user1':'bjinx', 'id':200}

[17] print(dic1)

     {'user1': 'abohr', 'id': 100}
```

Dictionaries in Python

```
[18] dic1.get('id') # Get values according to the key

     100

[19] dic3 = {'customer1':dic1,'customer2':dic2} # create a dictionary using other dictionaries

[20] dic3

     {'customer1': {'user1': 'abohr', 'id': 100},
      'customer2': {'user1': 'bjinx', 'id': 200}}
```

Diccionarios: acceder, modificar y agregar por clave

EJEMPLO

```
[1] estudiante = {"nombre": "Ana", "edad": 20, "materias": ["IA", "BD"]}
    estudiante["nombre"]
```

```
'Ana'
```

```
[2] estudiante["edad"] = 21            # modifica (mutable)
    estudiante["ciudad"] = "Guayaquil"  # agrega una clave
    estudiante
```

```
{'nombre': 'Ana', 'edad': 21, 'materias': ['IA', 'BD'], 'ciudad': 'Guayaquil'}
```

```
[3] estudiante.get("telefono", "No registrado")
```

```
'No registrado'
```

get() evita el error si la clave no existe.

```
[4] d = {"a": 1, "a": 2}   # clave repetida
    d
```

```
{'a': 2}
```

Las claves son únicas: se queda el último valor.

```
[5] estudiante[0]
```

```
KeyError: 0
```

No hay índices: se accede por clave, no por posición.

Analogía: como una agenda de contactos: buscas por el **nombre** (clave) y obtienes el **número** (valor).

Practice: A messy dataset

1. Go to the link: https://oscarbaruffa.com/messy/
2. Analyze the questions shown in the Google Form
3. Open the responses in https://shorturl.at/sXsTA
4. Export the file as csv and analyze it with Python

Summary

- Basic concepts in Python
- Collections in Python
- Analysing a dataset

Week 4

Algoritmos y modelos, división del dataset y tipos de machine learning

22.09.2026 - 24.09.2026

Learning outcomes

- Distinguish the concept of algorithm and model in the context of machine learning
- Describe the different datasets employed to build a machine learning model
- List the machine learning techniques
- Define the concept of supervised learning
- Define the concept of unsupervised learning

Algorithms in machine learning

- Procedure that is performed on a set of data to create a model.
- Tasks they perform: pattern recognition, learning from data.
- Some examples of algorithms:
  - Linear regression
  - Logistic regression
  - Neural networks

Model in machine learning

- Output of an algorithm after it has been executed on a dataset.
- Mathematical representation of what has been learned using an algorithm.
- The generation of a model depends on the data that an algorithm uses to learn.

Algoritmo vs. modelo: el mismo algoritmo, distintos datos

EJEMPLO

Datos A: casas en una ciudad

m^2	precio ($)
60	80 000
80	100 000
100	120 000

Datos B: casas en otra ciudad

m^2	precio ($)
60	87 000
80	111 000
100	135 000

Algoritmo:
regresión lineal
(el procedimiento para aprender)

→

Modelo A

```
precio = 1000 · m² + 20 000
```

Modelo B

```
precio = 1200 · m² + 15 000
```

Analogía: el algoritmo es la **receta**, los datos son los **ingredientes** y el modelo es el **plato terminado**. Con otros ingredientes, la misma receta da otro plato. Para predecir una casa de 90 m^2 usas el modelo: A → $110 000.

Machine learning algorithms

- Models are created from a set of data that allows patterns to be found and new predictions to be made.
- The models created must be evaluated with a different set of data to assess their performance in new cases.

Machine learning techniques

- Regression
- Classification
- Clustering
- Associations
- Anomaly detection
- Sequence mining
- Dimension reduction
- Recommendation systems

Una situación real para cada técnica

EJEMPLO

Regresión

Predecir el **precio** de una casa en dólares.

Clasificación

Decidir si un correo es **spam o no spam**.

Clustering

Agrupar clientes con hábitos de compra parecidos, sin grupos definidos antes.

Asociaciones

«Quien compra pan suele comprar mantequilla» → ponerlos cerca en la góndola.

Detección de anomalías

Una compra de $3 000 a las 3:00 a. m. en otro país con tu tarjeta.

Minería de secuencias

Patrones en orden: quien ve el producto, luego el carrito, luego el envío… ¿abandona?

Reducción de dimensión

Resumir 50 preguntas de una encuesta en 2 factores para poder graficarlas.

Sistemas de recomendación

Spotify: «Porque escuchaste a este artista…».

Pista: si la respuesta es un **número** → regresión; si es una **categoría** → clasificación; si **no hay respuesta conocida** y buscas grupos → clustering.

Splitting the dataset

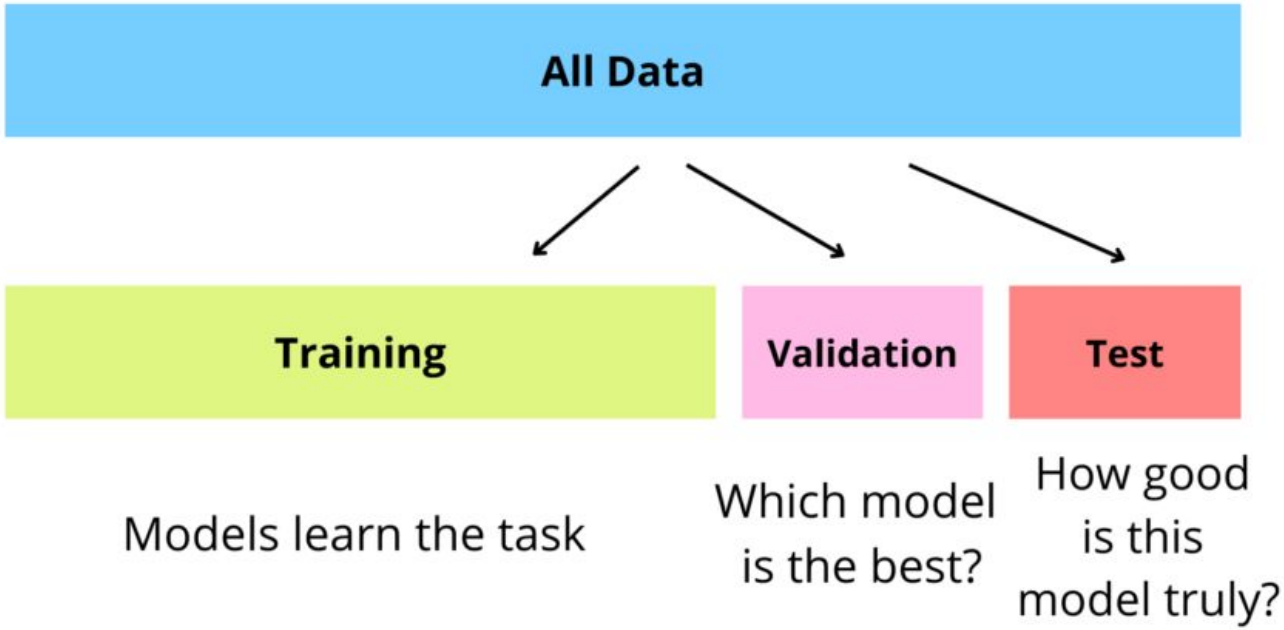
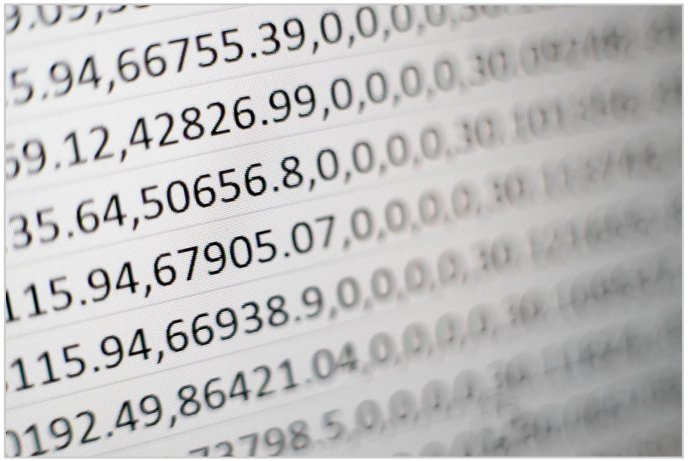


Image source: Galaxy Inferno

Training dataset

- Data used to train a model.
- Data previously analyzed and “cleaned.”
- The quality of the model will depend on the quality of the data.



Validation dataset

- Data that has similar characteristics to the training dataset (distribution, patterns).
- Used to verify the accuracy and error of the model before testing it on the test dataset.
- Facilitates the tuning of the model's hyperparameters.

Test dataset

- Data used to test the predictions of the generated model with new cases.
- Using this data set allows us to know the accuracy of the model once it has been adjusted with the validation data set.

Entrenamiento, validación y prueba: la analogía del estudiante

EJEMPLO

Entrenamiento · 700 fotos (70 %)	Validación · 150	Prueba · 150

Dataset de 1 000 fotos de perros y gatos para entrenar un clasificador.

Entrenamiento = deberes y clases

El estudiante practica con ejercicios **resueltos**. El modelo aprende patrones con datos etiquetados.

Validación = simulacro

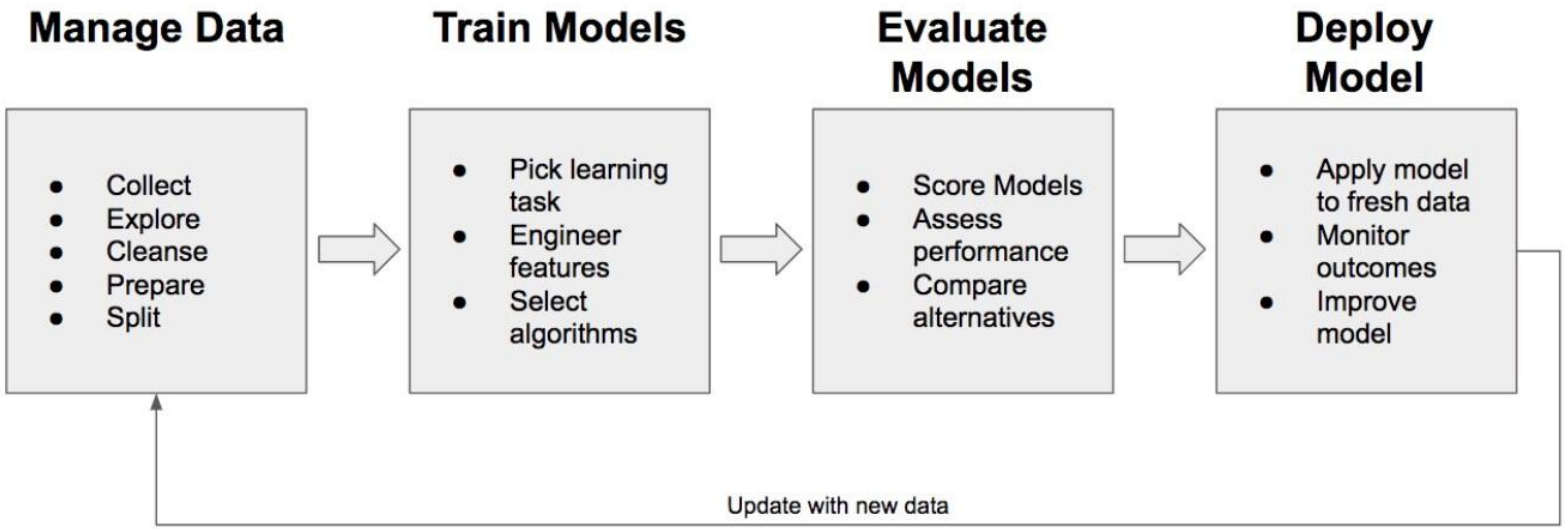
Una prueba de práctica para **ajustar cómo estudia** (más horas, otro método). El modelo ajusta sus hiperparámetros y se elige el mejor.

Prueba = examen final

Preguntas **nunca vistas**. Mide qué tan bien aprendió de verdad. Ej.: acierta 138 de 150 fotos → 92 %.

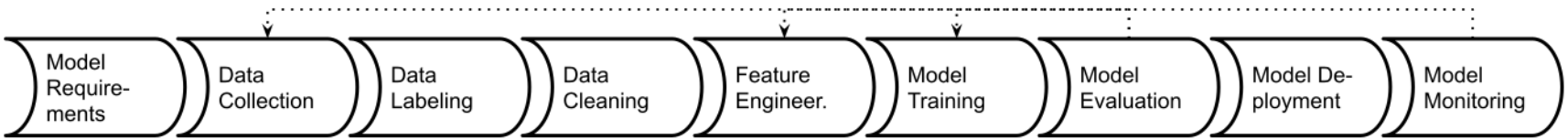
Si el examen tuviera las mismas preguntas de los deberes, solo medirías **memoria**. Por eso los datos de prueba deben ser **distintos** a los de entrenamiento.

Steps to create a machine learning model



AI Artisan. Copyright Maurice Chang, 2017.

Steps to build and deploy a machine learning model



Fuente: Machine Learning in Production/AI Engineering • Christian Kaestner & Eunsuk Kang, Carnegie Mellon University

Los pasos aplicados: app que predice el tiempo de entrega

EJEMPLO

1. Manage data

Recolectar 50 000 pedidos pasados (distancia, hora, lluvia, tiempo real). Limpiar pedidos cancelados y tiempos negativos. Dividir 70/15/15.

2. Train models

Tarea: **regresión** (el tiempo es un número). Features: distancia, hora pico, lluvia. Probar varios algoritmos.

3. Evaluate

Comparar el error: el modelo A se equivoca ±9 min y el B ±4 min → se elige B.

4. Deploy

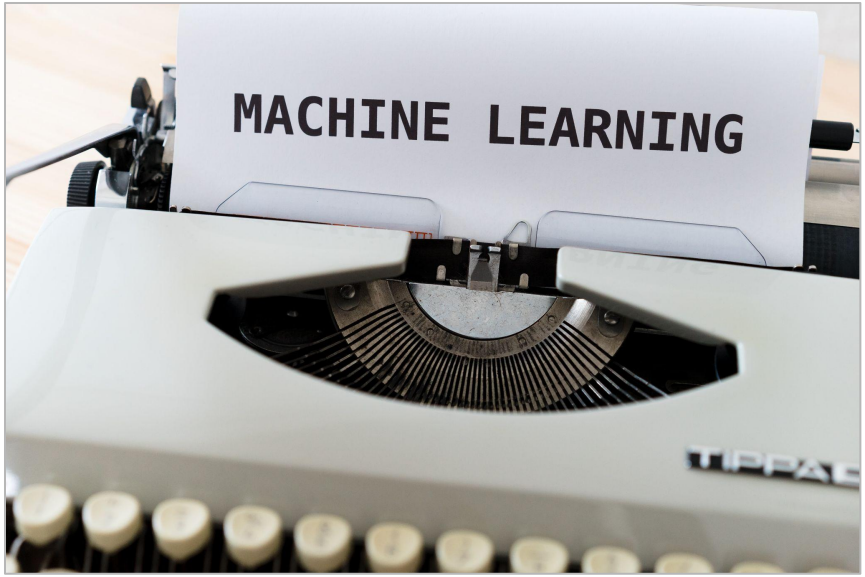
La app muestra «Llega en 25 min». Se monitorean los pedidos reales para detectar errores.

Nuevos pedidos (ej.: abre un local nuevo) → Se reentrena con datos frescos → Vuelve al paso 1

Es un **ciclo**, no una línea recta: un modelo en producción se actualiza cuando los datos cambian.

Types of machine learning

- Supervised learning
- Unsupervised learning
- Reinforcement learning



Supervised learning

- Using a labeled training data set
- Training data contains both input and output (label) data
- Finding a function that takes the input data and assigns an output label

Types of supervised learning

- Classification
  - The process of predicting class labels or categories
- Regression
  - The process of predicting continuous values

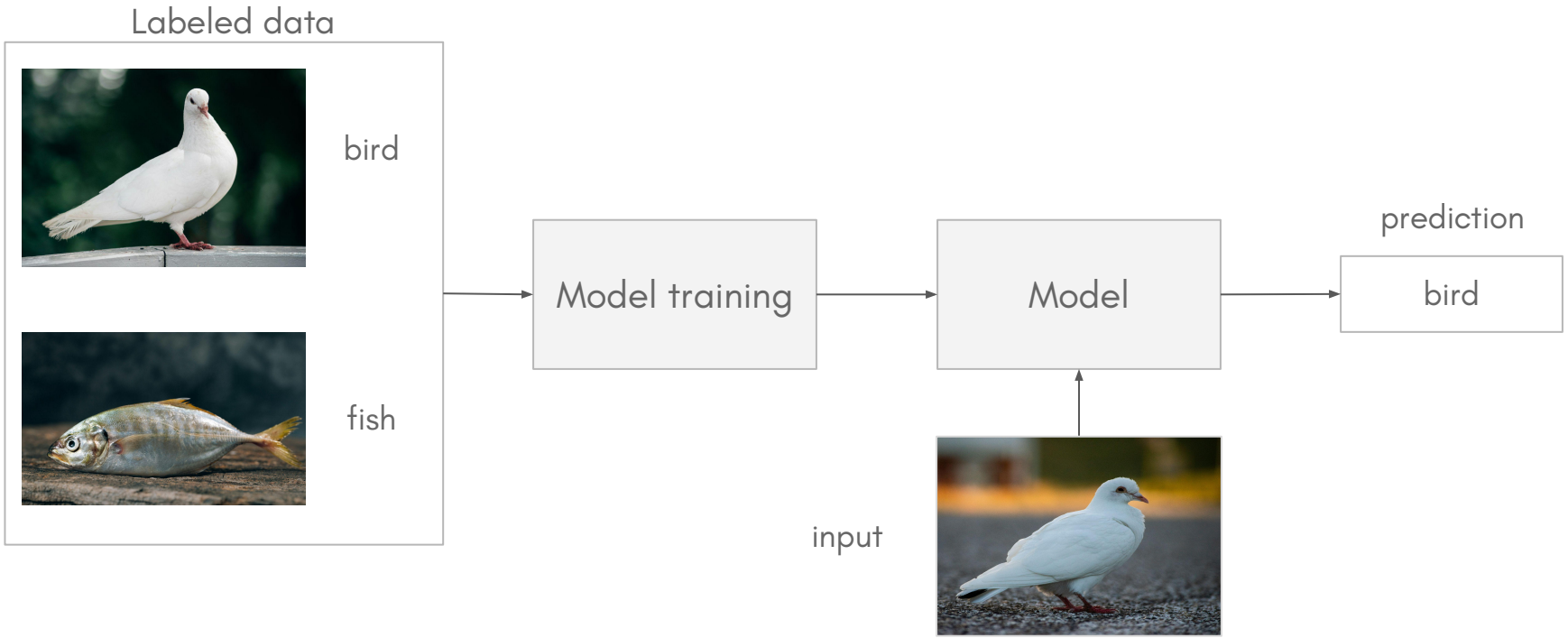
¿Clasificación o regresión? Mira el tipo de respuesta

EJEMPLO

Situación	Clasificación (categoría)	Regresión (número continuo)
Casas	¿Se venderá en menos de un mes? → **Sí / No**	¿A qué precio se venderá? → **$85 400**
Clima	¿Lloverá mañana? → **Sí / No**	¿Cuántos milímetros lloverá? → **12.5 mm**
Estudiantes	¿Aprobará el curso? → **Aprueba / Reprueba**	¿Qué nota sacará? → **78.5**
Correo	¿Es spam? → **Spam / No spam**	¿Cuántos correos llegarán mañana? → **143**
Fotos	¿Qué animal es? → **perro / gato / ave**	¿Cuántos años tiene el perro? → **4.2 años**

Ambas son **aprendizaje supervisado**: se entrenan con ejemplos que ya tienen la respuesta (etiqueta). Lo único que cambia es **el tipo de etiqueta**.

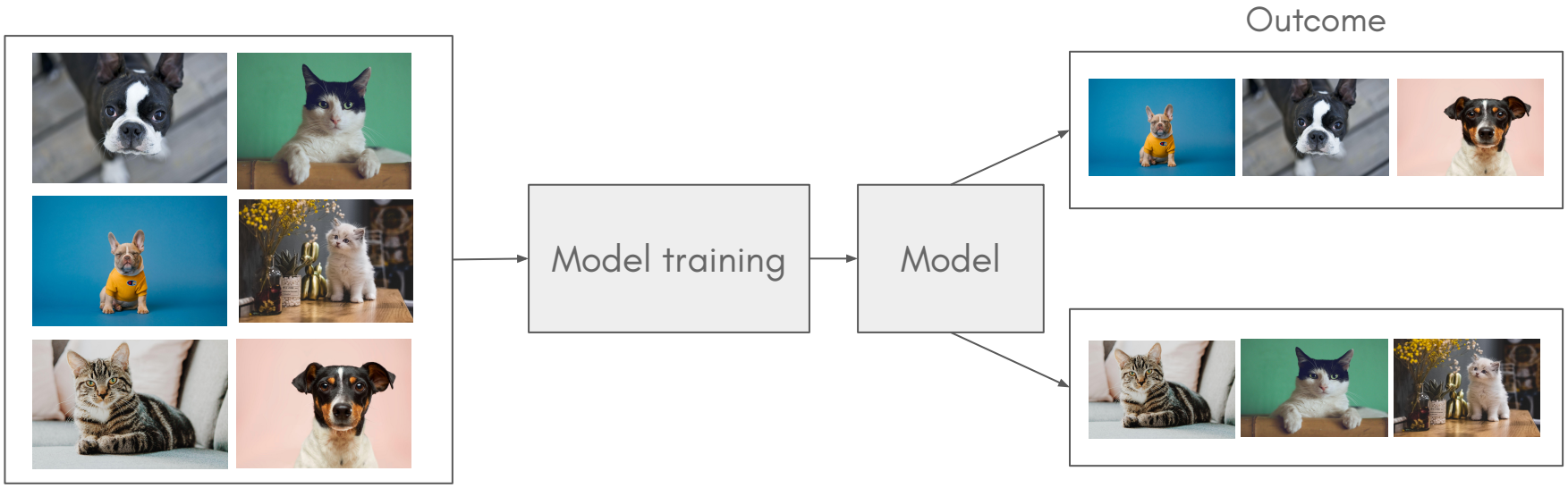
Supervised learning



Unsupervised learning

- Training the model with unlabeled data, which can result in less accurate predictions.
- This type of learning is used to group data into categories and detect anomalies.

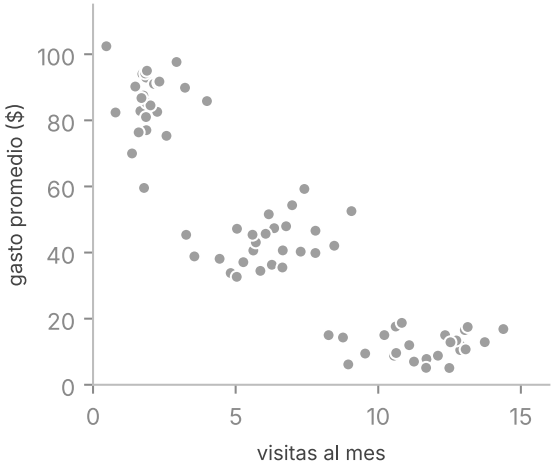
Unsupervised learning



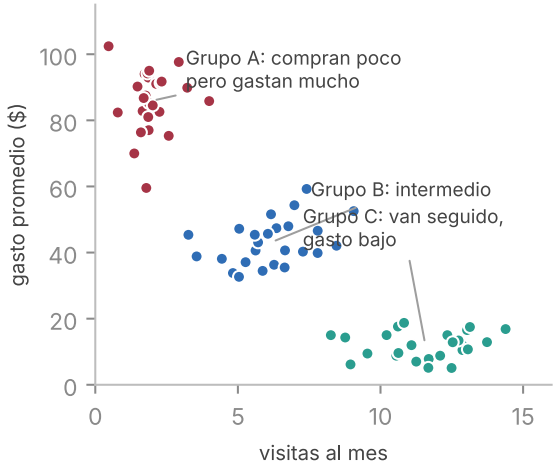
Aprendizaje no supervisado: segmentar clientes

EJEMPLO

Antes: datos SIN etiquetas



Después: el modelo encontró 3 grupos



Entrada

75 clientes con dos datos: visitas al mes y gasto promedio. **Nadie dice a qué grupo pertenece cada uno.**

Salida

El modelo forma 3 grupos por similitud. Las personas los interpretan y les ponen nombre.

Uso: una campaña distinta para cada grupo. **Anomalías:** un cliente lejos de todos los grupos (p. ej., 30 visitas y $500 de gasto) sería un caso raro para revisar.

Reinforcement learning

- Type of learning that does not require data for training.
- Learning occurs through the experience acquired by an agent in its environment. The agent observes the environment and responds with some action. Depending on the action, a reward is given, so the objective is to associate the observation with the action (policy) that leads to a reward.

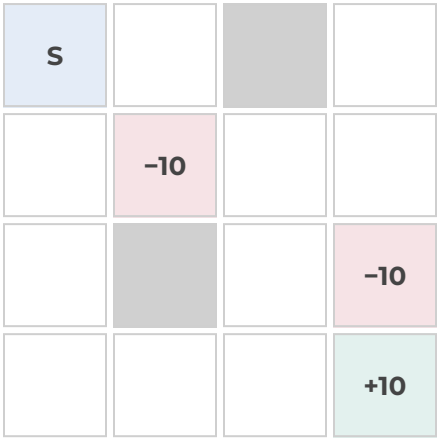
Reinforcement learning - explanation



https://youtu.be/spfpBrBjntg

Aprendizaje por refuerzo: un robot en un laberinto

EJEMPLO



S = salida del robot · gris = muro · cada paso cuesta −1

Término	En el ejemplo
Agente	El robot que toma decisiones.
Entorno	El laberinto.
Acción	Moverse ↑ ↓ ← →.
Recompensa	+10 al llegar a la meta, −10 en una trampa, −1 por paso.
Política	La regla aprendida: «en esta casilla, conviene ir a la derecha».

Intento 1

Se mueve al azar, cae en trampas: recompensa total **−27**.

Intento 500

Va directo por el camino más corto: **+4**.

No usó un dataset: aprendió por **prueba y error**, como un perro que aprende a sentarse con premios.

Summary

- Algorithms and models
- Splitting a dataset
- Types of machine learning

Week 5

Módulos de Python y exploración de datos con Pandas

29.09.2026 - 01.10.2026

Learning outcomes

- Define the concept of module in Python
- Describe the main functions used for data exploration in Pandas
- Explore a real dataset by using the learned functions in Pandas

Modules in Python

- Python includes a number of libraries created for different purposes, for example, libraries for calculating statistics or performing mathematical operations.
- The libraries must be imported to be used in code. For this purpose, the `import` keyword is used.

```
import module

import module as alias
```

Módulos: importar herramientas listas para usar

EJEMPLO

```
[1] import math
    math.sqrt(16)
```

```
4.0
```

Se usa con el nombre del módulo: math.sqrt().

```
[2] import statistics as st
    st.mean([85, 92, 78])
```

```
85
```

Con alias: escribes st en lugar de statistics.

Analogía: un módulo es una **caja de herramientas**. `import` la trae a tu mesa; el alias es un apodo corto.

```
[3] from math import pi
    round(pi, 4)
```

```
3.1416
```

from … import trae solo lo que necesitas.

```
[4] sqrt(25)
```

```
NameError: name 'sqrt' is not defined
```

Sin importar (o sin el prefijo), Python no conoce la función.

Módulo	Para qué
math	operaciones matemáticas
statistics	media, mediana…
pandas (pd)	tablas de datos
numpy (np)	arreglos numéricos

Pandas

- A Python library used for data handling and analysis.
- Use of data structures to facilitate data manipulation and processing.
- Used in Data Science and Machine Learning.

Pandas

- Installation

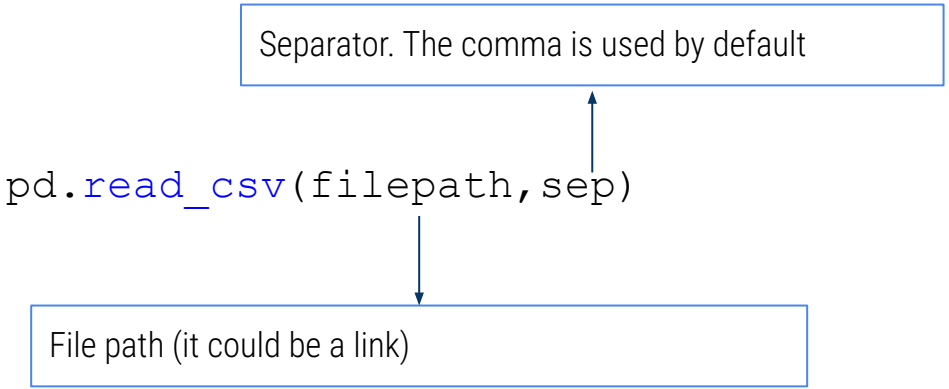
https://pandas.pydata.org/docs/getting_started/install.html

- Using the library in Python

```
import pandas as pd
```

Pandas - the read_csv() method

- A method used to read a file containing comma-separated values (csv).
- It returns a DataFrame which is the data structure used in Pandas, a table representation, which groups values into columns and uses indexing.



Documentation: Pandas read_csv method

read_csv(): de archivo de texto a DataFrame

EJEMPLO

Contenido de `notas.csv`

```
nombre,edad,ciudad,nota
Ana,20,Guayaquil,85
Luis,22,Quito,67
María,19,Guayaquil,92
Carlos,21,Cuenca,74
```

La 1.ª fila son los **encabezados**; cada coma separa una columna.

```
[1] import pandas as pd
    df = pd.read_csv("notas.csv")
    df
```

	nombre	edad	ciudad	nota
0	Ana	20	Guayaquil	85
1	Luis	22	Quito	67
2	María	19	Guayaquil	92
3	Carlos	21	Cuenca	74

¿Y si el CSV viene de Excel en español? Suele usar **punto y coma**: hay que indicar `sep=";"`

```
[2] pd.read_csv("notas_excel.csv").columns
```

```
Index(['nombre;edad;ciudad;nota'], dtype='object')
```

Archivo separado por ; leído con la coma por defecto: todo queda en una sola columna.

```
[3] pd.read_csv("notas_excel.csv", sep=";").columns
```

```
Index(['nombre', 'edad', 'ciudad', 'nota'], dtype='object')
```

Salary dataset

- Go to the link: https://oscarbaruffa.com/messy/
- Link to the dataset: https://shorturl.at/Xgq2a

Common methods in Pandas

Method	Description
df.info()	Returns information about the dataframe.
df.describe()	Returns a summary with the basic statistics of the dataframe (min, max, avg, count, std.)
df[column].unique()	Returns the unique values found in the dataframe according to the indicated column.
df.count()	Returns the number of non-empty cells found in the dataframe.

df.info() y df.count() con un DataFrame pequeño

EJEMPLO

DataFrame `df` de ejemplo

	nombre	edad	ciudad	nota
0	Ana	20	Guayaquil	85.0
1	Luis	22	Quito	67.0
2	María	19	Guayaquil	92.0
3	Carlos	21	Cuenca	74.0
4	Sofía	20	Guayaquil	NaN
5	Diego	23	Quito	58.0

NaN = valor vacío (Sofía no tiene nota). **object** = texto; **int64** = enteros; **float64** = decimales.

`[1]` `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 4 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   nombre  6 non-null      object
 1   edad    6 non-null      int64
 2   ciudad  6 non-null      object
 3   nota    5 non-null      float64
dtypes: float64(1), int64(1), object(2)
memory usage: 324.0+ bytes
```

Filas, columnas, tipo de dato y cuántos valores NO vacíos hay en cada columna.

`[2]` `df.count()`

```
nombre    6
edad      6
ciudad    6
nota      5
dtype: int64
```

Solo el conteo de no vacíos: nota tiene 5 porque falta una.

df.describe() y unique(): cómo leer los resultados

EJEMPLO

```
[1] df.describe()
```

	edad	nota
count	6.000000	5.000000
mean	20.833333	75.200000
std	1.471960	13.627179
min	19.000000	58.000000
25%	20.000000	67.000000
50%	20.500000	74.000000
75%	21.750000	85.000000
max	23.000000	92.000000

Solo columnas numéricas. count = 5 en nota porque el NaN no cuenta.

```
[2] df["ciudad"].unique()
```

```
array(['Guayaquil', 'Quito', 'Cuenca'], dtype=object)
```

Valores distintos, sin repetir.

Fila	Significado (columna nota)
count	5 notas registradas
mean	promedio: 376 / 5 = 75.2
std	dispersión: qué tan separadas están
min / max	nota más baja (58) y más alta (92)
25 % / 50 % / 75 %	percentiles: 50 % = mediana (74)

Úsalo para detectar errores rápido: una edad **max = 200** o una nota negativa revelan datos mal ingresados.

Exploring a dataset with Pandas

```
df = pd.read_csv("salary_dataset.csv")
df
```

	Timestamp	How old are you?	What industry do you work in?	Job title	If needs additional context, please clarify here:	If time or hourly, please enter an annualized equivalent -- what you would earn if you worked the job 40 hours a week, 52 weeks a year.)	bonuses or overtime in an average year)? Please only include monetary compensation here, not the value of benefits.	Please indicate the currency	If please indicate the currency here:	If needs additional context, please provide it here:	country do you work in?	In the U.S., what state do you work in?
0	4/27/2021 11:02:10	25-34	Education (Higher Education)	Research and Instruction Librarian	NaN	55,000	0.0	USD	NaN	NaN	United States	Massachusetts
1	4/27/2021 11:02:22	25-34	Computing or Tech	Change & Internal Communications Manager	NaN	54,600	4000.0	GBP	NaN	NaN	United Kingdom	NaN
2	4/27/2021 11:02:38	25-34	Accounting, Banking & Finance	Marketing Specialist	NaN	34,000	NaN	USD	NaN	NaN	US	Tennessee
3	4/27/2021 11:02:41	25-34	Nonprofits	Program Manager	NaN	62,000	3000.0	USD	NaN	NaN	USA	Wisconsin
4	4/27/2021 11:02:42	25-34	Accounting, Banking & Finance	Accounting Manager	NaN	60,000	7000.0	USD	NaN	NaN	US	South Carolina
...	...	...	...	...	...	...	...	...	...	...	...	...

Changing the headers to the dataframe

```
rename_mapping = {
    'Timestamp': 'timestamp',
    'How old are you?': 'age',
    'What industry do you work in?': 'industry',
    'Job title': 'job_title',
    'If your job title needs additional context, please clarify here:': 'job_title_context',
    "What is your annual salary? (You'll indicate the currency in a later question. If you are part-time or hourly, pl
    'How much additional monetary compensation do you get, if any (for example, bonuses or overtime in an average year
    'Please indicate the currency': 'currency',
    'If "Other," please indicate the currency here: ': 'currency_other',
    'If your income needs additional context, please provide it here:': 'income_context',
    'What country do you work in?': 'country',
    "If you're in the U.S., what state do you work in?": 'state',
    'What city do you work in?': 'city',
    'How many years of professional work experience do you have overall?': 'total_experience_years',
    'How many years of professional work experience do you have in your field?': 'field_experience_years',
    'What is your highest level of education completed?': 'education',
    'What is your gender?': 'gender',
    'What is your race? (Choose all that apply.)': 'race'
}
```

```
df = df.rename(columns=rename_mapping)
df.columns
```

```
Index(['timestamp', 'age', 'industry', 'job_title', 'job_title_context',
       'annual_salary', 'additional_comp', 'currency', 'currency_other',
       'income_context', 'country', 'state', 'city', 'total_experience_years',
       'field_experience_years', 'education', 'gender', 'race'],
      dtype='object')
```

rename(): encabezados cortos y fáciles de usar

EJEMPLO

```
[1] df.columns.tolist()
```

```
['Nombre del estudiante', 'Edad (años)', 'Ciudad de origen']
```

```
[2] mapeo = {
        "Nombre del estudiante": "nombre",
        "Edad (años)": "edad",
        "Ciudad de origen": "ciudad",
    }
    df = df.rename(columns=mapeo)
    df
```

	nombre	edad	ciudad
0	Ana	20	Guayaquil
1	Luis	22	Quito

¿Por qué?

Los formularios (como Google Forms) usan la pregunta completa como encabezado. Escribir `df["Edad (años)"]` cada vez es incómodo y propenso a errores.

El diccionario va **{"nombre viejo": "nombre nuevo"}**. Se guarda con `df = df.rename(…)`; si no, el cambio se pierde.

Show information of the dataframe

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 28226 entries, 0 to 28225
Data columns (total 18 columns):
 #   Column                  Non-Null Count  Dtype  
---  ------                  --------------  -----  
 0   timestamp               28226 non-null  object 
 1   age                     28226 non-null  object 
 2   industry                28140 non-null  object 
 3   job_title               28221 non-null  object 
 4   job_title_context       7293 non-null   object 
 5   annual_salary           28226 non-null  object 
 6   additional_comp         20850 non-null  float64
 7   currency                28226 non-null  object 
 8   currency_other          225 non-null    object 
 9   income_context          3053 non-null   object 
 10  country                 28224 non-null  object 
 11  state                   23148 non-null  object 
 12  city                    28141 non-null  object 
 13  total_experience_years  28226 non-null  object 
 14  field_experience_years  28226 non-null  object 
 15  education               27985 non-null  object 
 16  gender                  28040 non-null  object 
 17  race                    28029 non-null  object 
dtypes: float64(1), object(17)
memory usage: 3.9+ MB
```

Basic statistics of the dataframe

```
df.describe()
```

	annual_salary	additional_comp
count	2.822600e+04	2.085000e+04
mean	3.711248e+05	1.822032e+04
std	3.613302e+07	8.324853e+05
min	0.000000e+00	0.000000e+00
25%	5.400000e+04	0.000000e+00
50%	7.500000e+04	2.000000e+03
75%	1.094375e+05	1.000000e+04
max	6.000070e+09	1.200000e+08

Head function

- Returns the first records of the data frame. By default, it returns the first 5.
- You can specify the elements to display in the function, for example: df.head(10).

```
df.head()
```

	timestamp	age	industry	job_title	job_title_context	annual_salary	additional_comp	currency	currency_other	income_context	country	state	city
0	4/27/2021 11:02:10	25-34	Education (Higher Education)	Research and Instruction Librarian	NaN	55000	0.0	USD	NaN	NaN	United States	Massachusetts	Boston
1	4/27/2021 11:02:22	25-34	Computing or Tech	Change & Internal Communications Manager	NaN	54600	4000.0	GBP	NaN	NaN	United Kingdom	NaN	Cambridge
2	4/27/2021 11:02:38	25-34	Accounting, Banking & Finance	Marketing Specialist	NaN	34000	NaN	USD	NaN	NaN	US	Tennessee	Chattanooga
3	4/27/2021 11:02:41	25-34	Nonprofits	Program Manager	NaN	62000	3000.0	USD	NaN	NaN	USA	Wisconsin	Milwaukee
4	4/27/2021 11:02:42	25-34	Accounting, Banking & Finance	Accounting Manager	NaN	60000	7000.0	USD	NaN	NaN	US	South Carolina	Greenville

Tail function

- Returns the last records in the data frame. By default, it returns the first 5.
- You can specify the elements to display in the function, for example: df.tail(8).

```
df.tail()
```

	timestamp	age	industry	job_title	job_title_context	annual_salary	additional_comp	currency	currency_other	income_context	country	state	city	t
28221	6/2/2026 14:06:08	25-34	Accounting, Banking & Finance	Accountant	NaN	53000	6000.0	GBP	NaN	NaN	UK	NaN	London	
28222	6/22/2026 1:24:59	18-24	Education (Higher Education)	student	NaN	0	NaN	Other	PKR	NaN	pakistan	NaN	faisalabad	
28223	7/25/2026 5:10:49	18-24	Education (Primary/Secondary)	English Teacher	NaN	31200	NaN	Other	MYR	NaN	Malaysia	NaN	Pahang	
28224	7/27/2026 11:44:14	35-44	Retail	purchasing coordinator	NaN	12000	0.0	USD	NaN	NaN	romania	NaN	bucharest	
28225	9/17/2026 20:18:08	25-34	Computing or Tech	Arbol	NaN	21	NaN	USD	NaN	NaN	Ecuador	Alabama	Texas	

head() y tail(): ver el inicio y el final

EJEMPLO

DataFrame completo (8 filas)

	nombre	nota
0	Ana	85
1	Luis	67
2	María	92
3	Carlos	74
4	Sofía	88
5	Diego	58
6	Valeria	79
7	Pedro	95

```
[1] df.head(3)
```

	nombre	nota
0	Ana	85
1	Luis	67
2	María	92

```
[2] df.tail(2)
```

	nombre	nota
6	Valeria	79
7	Pedro	95

Sin número: `head()` y `tail()` muestran ***5*** filas.

El **índice** (0, 1, 2…) se conserva: en tail ves 6 y 7, no 0 y 1.

Con 28 000 filas (como el dataset de salarios) es la forma rápida de dar un vistazo.

Delete a column

```
df.drop(['timestamp'],axis=1, inplace=True)
```

```
df
```

	age	industry	job_title	job_title_context	annual_salary	additional_comp	currency	currency_other	income_context	country	state	
0	25-34	Education (Higher Education)	Research and Instruction Librarian	NaN	55000	0.0	USD	NaN	NaN	United States	Massachusetts	
1	25-34	Computing or Tech	Change & Internal Communications Manager	NaN	54600	4000.0	GBP	NaN	NaN	United Kingdom	NaN	C
2	25-34	Accounting, Banking & Finance	Marketing Specialist	NaN	34000	NaN	USD	NaN	NaN	US	Tennessee	Ch
3	25-34	Nonprofits	Program Manager	NaN	62000	3000.0	USD	NaN	NaN	USA	Wisconsin	I
4	25-34	Accounting, Banking & Finance	Accounting Manager	NaN	60000	7000.0	USD	NaN	NaN	US	South Carolina	
...	...	...	...	...	...	...	...	...	...	...	...	

drop(): borrar columnas o filas

EJEMPLO

```
[1] df.drop(["edad"], axis=1)
```

	nombre	nota
0	Ana	85.0
1	Luis	67.0
2	María	92.0

axis=1 → borra una COLUMNA.

```
[2] df.drop([0], axis=0)
```

	nombre	edad	nota
1	Luis	22	67.0
2	María	19	92.0

axis=0 → borra una FILA (por su índice).

axis	Borra...	Recuerda
1	columnas	↔ a lo ancho
0	filas	↕ a lo largo

```
[3] df.columns.tolist()
```

```
['nombre', 'edad', 'nota']
```

¡df sigue intacto! drop devolvió una copia.

```
[4] df.drop(["edad"], axis=1, inplace=True)
    df.columns.tolist()
```

```
['nombre', 'nota']
```

inplace=True modifica el df original.

Dos formas equivalentes de guardar el cambio:

```
df = df.drop([...], axis=1)
df.drop([...], axis=1, inplace=True)
```

Check null values with isna()

```
df.isna()
```

	age	industry	job_title	job_title_context	annual_salary	additional_comp	currency	currency_other	income_context	country	state	city
0	False	False	False	True	False	False	False	True	True	False	False	False
1	False	False	False	True	False	False	False	True	True	False	True	False
2	False	False	False	True	False	True	False	True	True	False	False	False
3	False	False	False	True	False	False	False	True	True	False	False	False
4	False	False	False	True	False	False	False	True	True	False	False	False
...	...	...	...	...	...	...	...	...	...	...	...	...
28221	False	False	False	True	False	False	False	True	True	False	True	False
28222	False	False	False	True	False	True	False	False	True	False	True	False
28223	False	False	False	True	False	True	False	False	True	False	True	False
28224	False	False	False	True	False	False	False	True	True	False	True	False
28225	False	False	False	True	False	True	False	True	True	False	False	False

28226 rows × 17 columns

Check null values with isnull()

```
df.isnull()
```

	age	industry	job_title	job_title_context	annual_salary	additional_comp	currency	currency_other	income_context	country	state	city	total_experience_years
0	False	False	False	True	False	False	False	True	True	False	False	False	False
1	False	False	False	True	False	False	False	True	True	False	True	False	False
2	False	False	False	True	False	True	False	True	True	False	False	False	False
3	False	False	False	True	False	False	False	True	True	False	False	False	False
4	False	False	False	True	False	False	False	True	True	False	False	False	False
...	...	...	...	...	...	...	...	...	...	...	...	...	...
28221	False	False	False	True	False	False	False	True	True	False	True	False	False
28222	False	False	False	True	False	True	False	False	True	False	True	False	False
28223	False	False	False	True	False	True	False	False	True	False	True	False	False
28224	False	False	False	True	False	False	False	True	True	False	True	False	False
28225	False	False	False	True	False	True	False	True	True	False	False	False	False

28226 rows × 17 columns

Check total null values

```
df.isnull().sum()
```

	0
age	0
industry	86
job_title	5
job_title_context	20933
annual_salary	0
additional_comp	7376
currency	0
currency_other	28001
income_context	25173
country	2
state	5078

Valores vacíos: isna(), isnull() y .sum()

EJEMPLO

DataFrame `df` con vacíos

	nombre	edad	nota
0	Ana	20.0	85.0
1	Luis	NaN	67.0
2	None	19.0	NaN
3	Carlos	21.0	NaN

`None` y `NaN` son vacíos para Pandas.

```
[1] df.isna()
```

	nombre	edad	nota
0	False	False	False
1	False	True	False
2	True	False	True
3	False	False	True

True = vacío. isnull() da exactamente lo mismo.

```
[2] df.isnull().sum()
```

```
nombre    1
edad      1
nota      2
dtype: int64
```

Suma los True por columna: el total de vacíos.

¿Por qué funciona .sum()? Python trata **True como 1** y **False como 0**. Sumar la columna = contar los vacíos.

`isna()` e `isnull()` son **sinónimos**: usa el que prefieras.

En el dataset de salarios, **currency_other** tiene 28 001 vacíos de 28 226: casi nadie llenó esa pregunta.

Select a column

```
df['industry']
```

	industry
0	Education (Higher Education)
1	Computing or Tech
2	Accounting, Banking & Finance
3	Nonprofits
4	Accounting, Banking & Finance
...	...
28221	Accounting, Banking & Finance
28222	Education (Higher Education)
28223	Education (Primary/Secondary)
28224	Retail
28225	Computing or Tech

28226 rows × 1 columns

Value counts of a column

```
df['industry'].value_counts()
```

industry	count
Computing or Tech	4725
Education (Higher Education)	2481
Nonprofits	2421
Health care	1905
Government and Public Administration	1894
...	...
Pet care industry (dog training/walking)	1
Academic Medicine	1
STEM research	1
Biotech R&D	1
Oil and gas exploration	1

1225 rows × 1 columns

Check unique values of a column

```
df['country'].unique()
```

```
array(['United States', 'United Kingdom', 'US', 'USA', 'Canada',
       'United Kingdom ', 'usa', 'UK', 'Scotland ', 'U.S.',
       'United States ', 'The Netherlands', 'Australia ', 'Spain', 'us',
       'Usa', 'England', 'finland', 'United States of America', 'France',
       'United states', 'Scotland', 'USA ', 'United states ', 'Germany',
       'UK ', 'united states', 'Ireland', 'India', 'Australia', 'Uk',
       'United States of America ', 'U.S. ', 'canada', 'Canada ', 'U.S>',
       'ISA', 'Argentina', 'Great Britain ', 'US ', 'United State',
       'U.S.A', 'Denmark', 'U.S.A.', 'America', 'Netherlands',
       'netherlands', 'England ', 'united states of america', 'Ireland ',
       'Switzerland', 'Netherlands ', 'Bermuda', 'Us',
       'The United States', 'United State of America', 'Germany ',
       'Malaysia', 'Mexico ', 'United Stated', 'South Africa ', 'Belgium',
       'Northern Ireland', 'u.s.', 'South Africa', 'UNITED STATES',
       'united States', 'Sweden', 'Hong Kong', 'Kuwait', 'Norway',
       'Sri lanka', 'Contracts', 'USA-- Virgin Islands', 'United Statws',
       'England/UK', 'U.S',
```

Remove spaces in column values

```
df['country']=df['country'].str.replace('United States','USA')
```

```
df['country'].unique()
```

```
array(['USA', 'United Kingdom', 'US', 'Canada', 'United Kingdom ', 'usa',
       'UK', 'Scotland ', 'U.S.', 'USA ', 'The Netherlands', 'Australia ',
       'Spain', 'us', 'Usa', 'England', 'finland', 'USA of America',
       'France', 'United states', 'Scotland', 'United states ', 'Germany',
       'UK ', 'united states', 'Ireland', 'India', 'Australia', 'Uk',
       'USA of America ', 'U.S. ', 'canada', 'Canada ', 'ISA',
       'Argentina', 'Great Britain ', 'US ', 'United State', 'U.S.A',
       'Denmark', 'U.S.A.', 'America', 'Netherlands', 'netherlands',
       'England ', 'united states of america', 'Ireland ', 'Switzerland',
       'Netherlands ', 'Bermuda', 'Us', 'The USA',
       'United State of America', 'Germany ', 'Malaysia', 'Mexico ',
       'United Stated', 'South Africa ', 'Belgium', 'Northern Ireland',
       'u.s.', 'South Africa', 'UNITED STATES', 'united States', 'Sweden',
       'Hong Kong', 'Kuwait', 'Norway', 'Sri lanka', 'Contracts',
       'USA-- Virgin Islands', 'United Statws', 'England/UK', 'U.S',
```

Limpiar una columna desordenada

EJEMPLO

```
[1] df["pais"].value_counts()
```

```
pais
Ecuador      2
ecuador      1
 Ecuador     1
EC           1
Perú         1
Name: count, dtype: int64
```

Antes: el mismo país aparece de 4 formas distintas.

```
[2] df["pais"] = df["pais"].str.strip()                  # quita espacios
    df["pais"] = df["pais"].str.replace("EC", "Ecuador")  # reemplaza
    df["pais"] = df["pais"].str.title()                  # Mayúscula inicial
    df["pais"].value_counts()
```

```
pais
Ecuador    5
Perú       1
Name: count, dtype: int64
```

Después: datos limpios y conteo correcto.

Método	Qué hace
`.str.strip()`	quita espacios al inicio y al final
`.str.replace(a, b)`	cambia el texto a por b
`.str.title()`	«ecuador» → «Ecuador»
`.str.lower()`	todo en minúsculas

Igual que «United States», «USA», «U.S.» en el dataset de salarios: para la computadora, **cada forma de escribir es un valor distinto**.

Selecting values from a dataframe by position

Dataframes use indexes to get the value in a specific row and column.

```
dataframe.iloc[idx_row,idx_col]
```

For example:

```
df.iloc[0,14]
```

```
'Master's degree'
```

Selecting values of a dataframe by position - use of iloc

```
df.iloc[0:5]
```

	age	industry	job_title	job_title_context	annual_salary	additional_comp	currency	currency_other	income_context	country	state	city	total_experience_years	field_experience_years	education	gender	race
0	25-34	Education (Higher Education)	Research and Instruction Librarian	NaN	55000	0.0	USD	NaN	NaN	USA	Massachusetts	Boston	5-7 years	5-7 years	Master's degree	Woman	White
1	25-34	Computing or Tech	Change & Internal Communications Manager	NaN	54600	4000.0	GBP	NaN	NaN	United Kingdom	NaN	Cambridge	8 - 10 years	5-7 years	College degree	Non-binary	White
2	25-34	Accounting, Banking & Finance	Marketing Specialist	NaN	34000	NaN	USD	NaN	NaN	US	Tennessee	Chattanooga	2 - 4 years	2 - 4 years	College degree	Woman	White
3	25-34	Nonprofits	Program Manager	NaN	62000	3000.0	USD	NaN	NaN	USA	Wisconsin	Milwaukee	8 - 10 years	5-7 years	College degree	Woman	White
4	25-34	Accounting, Banking & Finance	Accounting Manager	NaN	60000	7000.0	USD	NaN	NaN	US	South Carolina	Greenville	8 - 10 years	5-7 years	College degree	Woman	White

```
df.iloc[0:5,0:2]
```

	age	industry
0	25-34	Education (Higher Education)
1	25-34	Computing or Tech
2	25-34	Accounting, Banking & Finance
3	25-34	Nonprofits
4	25-34	Accounting, Banking & Finance

Show certain rows using iloc[]

Show rows from 0 al 14 for the column job_title

```
df['job_title'].iloc[0:15]
```

	job_title
0	Research and Instruction Librarian
1	Change & Internal Communications Manager
2	Marketing Specialist
3	Program Manager
4	Accounting Manager
5	Scholarly Publishing Librarian
6	Publishing Assistant
7	Librarian
8	Systems Analyst
9	Senior Accountant
10	Office Manager
11	Deputy Title IX Coordinator/ Assistant Directo...
12	Manager of Information Services
13	Legal Aid Staff Attorney
14	Patient care coordinator

iloc: seleccionar por posición (números)

EJEMPLO

Posiciones de `df`

	0: nombre	1: edad	2: ciudad	3: nota
0	Ana	20	**Guayaquil**	85
1	Luis	22	Quito	67
2	María	19	Guayaquil	92
…	…	…	…	…

`df.iloc[fila, columna]` usa solo **números de posición**, empezando en 0.

```
[1] df.iloc[0, 2]
```

```
'Guayaquil'
```

Fila 0, columna 2 (ciudad) → la ciudad de Ana.

```
[2] df.iloc[1:3]
```

	nombre	edad	ciudad	nota
1	Luis	22	Quito	67.0
2	María	19	Guayaquil	92.0

Filas 1 y 2 (el 3 no se incluye, como en slicing).

```
[3] df.iloc[:, [0, 3]]
```

	nombre	nota
0	Ana	85.0
1	Luis	67.0
2	María	92.0
3	Carlos	74.0
4	Sofía	NaN
5	Diego	58.0

Todas las filas (:), columnas 0 y 3.

Selecting rows based on a condition - use of loc

```
df.loc[df['country']=='Ecuador']
```

	age	industry	job_title	job_title_context	annual_salary	additional_comp	currency	currency_other	income_context	country	state	city
19005	25-34	Engineering or Manufacturing	Petroleum Engineer	NaN	84000	NaN	USD	NaN	NaN	Ecuador	NaN	Quito
28225	25-34	Computing or Tech	Arbol	NaN	21	NaN	USD	NaN	NaN	Ecuador	Alabama	Texas

loc: seleccionar por condición o por nombre

EJEMPLO

```
[1] df.loc[df["nota"] >= 70]
```

	nombre	edad	ciudad	nota
0	Ana	20	Guayaquil	85.0
2	María	19	Guayaquil	92.0
3	Carlos	21	Cuenca	74.0

Filtra las filas que cumplen la condición.

```
[2] df.loc[df["ciudad"] == "Guayaquil", ["nombre", "nota"]]
```

	nombre	nota
0	Ana	85.0
2	María	92.0
4	Sofía	NaN

Condición + solo algunas columnas (por nombre).

```
[3] df.loc[2, "nombre"]
'María'
```

Por etiqueta: fila con índice 2, columna «nombre».

	iloc	loc
Usa…	posiciones (números)	etiquetas y condiciones
Columnas	`[:, 3]`	`[:, "nota"]`
Filtrar	—	`df["nota"] ⩾ 70`

Sofía no aparece en el filtro: su nota es **NaN** y NaN ⩾ 70 es **False**.

En la clase: `df.loc[df['country']=='Ecuador']` devolvió las 2 personas de Ecuador.

Summary

- Modules in Python
- Introduction to Pandas
- Analysing a dataset with Pandas

Week 6

Variables dependientes e independientes, Matplotlib, NumPy e introducción a la regresión

06.10.2026 - 08.10.2026

Learning outcomes

- Distinguish the concept of dependent and independent variables.
- Describe the different methods used to create plots in Python.
- Describe the different methods used to work with arrays in Python.
- Describe the terms involved in the process of building a regression model.

Important concepts in machine learning

- Feature
  - Characteristics of the data. For instance: size, age, weight, height
- Label
  - A particular feature that is predicted by considering other features. For instance: predict an animal on a picture (e.g., cat, dog, turtle)
- Predictions
  - The output generated by the model (the guess)

Feature, label y predicción en una tabla

EJEMPLO

FEATURES (X) · características de entrada				LABEL (y)
m^2	habitaciones	años de antigüedad	¿tiene garaje?	precio ($)
60	2	15	no	78 000
95	3	5	sí	131 000
120	4	20	sí	142 000
80	3	10	sí	**? → predicción: 104 500**

Feature

Lo que **conoces** de la casa: cada columna de entrada.

Label

Lo que quieres **predecir**. En los datos de entrenamiento ya se conoce.

Predicción

La «adivinanza» del modelo para una casa nueva cuyo precio aún no se conoce.

Exercise

What will be the outputs in the three cases?

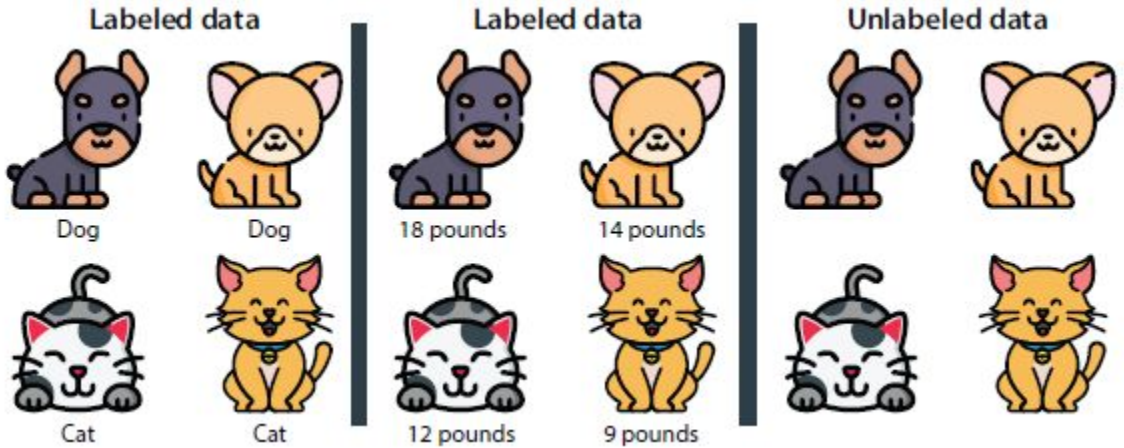


Image source: Grokking Machine Learning - Luis Serrano

Respuesta al ejercicio: ¿qué sale en cada caso?

EJEMPLO

Caso 1 · etiquetas «Dog / Cat»

Supervisado · Clasificación

El modelo aprende a responder una **categoría**. Ante un animal nuevo dice: **«Dog»** o **«Cat»**.

Caso 2 · etiquetas en libras

Supervisado · Regresión

La etiqueta es un **número** (el peso). Ante un animal nuevo predice algo como **«15 pounds»**.

Caso 3 · sin etiquetas

No supervisado · Clustering

No sabe qué es un perro o un gato. Solo **agrupa** los animales parecidos: «grupo 1» y «grupo 2».

Las **mismas imágenes** sirven para tres tareas distintas: lo que decide el tipo de aprendizaje es **qué etiqueta tienen** (categoría, número o ninguna).

Exercise

Determine if the following scenarios are examples of supervised and unsupervised learning:

1. A recommendation system on a social network that suggest friends to a user
2. A system in a news site that divided the news into topics
3. The autocomplete feature of Google to write sentences
4. A system in a company that detect fraudulent transactions with credit card.
5. A recommendation system that suggest products to buy considering the previous purchasing history of users

Respuestas sugeridas: ¿supervisado o no supervisado?

EJEMPLO

Escenario	Respuesta	¿Por qué?
1. Sugerir amigos en una red social	No supervisado	Agrupa usuarios parecidos (amigos en común, intereses). También válido supervisado si aprende de solicitudes aceptadas/rechazadas.
2. Dividir noticias en temas	No supervisado	Clustering: agrupa noticias por similitud sin temas etiquetados de antemano.
3. Autocompletado de Google	Supervisado	Aprende de millones de frases: dado el texto previo, la «etiqueta» es la palabra que sigue.
4. Detectar fraude con tarjeta	Supervisado	Clasificación con transacciones etiquetadas fraude / no fraude. También: no supervisado como detección de anomalías.
5. Recomendar productos por historial	Supervisado	El historial dice qué compró cada usuario (etiqueta: compró / no compró). También: no supervisado con reglas de asociación.

El libro del que viene el ejercicio (*Grokking Machine Learning*) acepta ambas respuestas si el razonamiento es correcto. La pregunta clave: **¿hay una etiqueta conocida que el modelo aprende a predecir?**

Dependent and independent variable

Consider the variable X to which one or more values of another variable Y correspond, we can say that Y is a function of X.

$$Y = f(X)$$

X → independent variable

Y → dependent variable

Variable independiente (X) y dependiente (Y)

EJEMPLO

X · independiente (la causa / entrada)	Y · dependiente (el efecto / salida)
Horas de estudio	Nota del examen
Temperatura del día	Helados vendidos
Metros cuadrados de una casa	Precio de la casa
Kilómetros recorridos en taxi	Valor a pagar

Y = f(X) con números

Taxi: `valor = 0.40 · km + 1.50`

Si X = 10 km → Y = 0.40 · 10 + 1.50 = **$5.50**

Prueba rápida: «Y depende de X». La nota depende de las horas de estudio ✔; las horas de estudio no dependen de la nota ✘.

Features

- Measurements made on the object being analyzed.
- In a data set, the features are the columns.

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2
...	...	...	...	...
145	6.7	3.0	5.2	2.3
146	6.3	2.5	5.0	1.9
147	6.5	3.0	5.2	2.0
148	6.2	3.4	5.4	2.3
149	5.9	3.0	5.1	1.8

Supervised learning

Input: X

Independent variables

Output: Y

Dependent variable

$$f(input) = output$$

A model is not entirely accurate so it always has an error

Supervised learning

Input: X

Independent variables

Output: Y

Dependent variable

$$f(input) + error = output$$

A model is not entirely accurate so it always has an error

Regression

- Process of predicting a continuous variable
- Two types of variables involved: dependent and independent

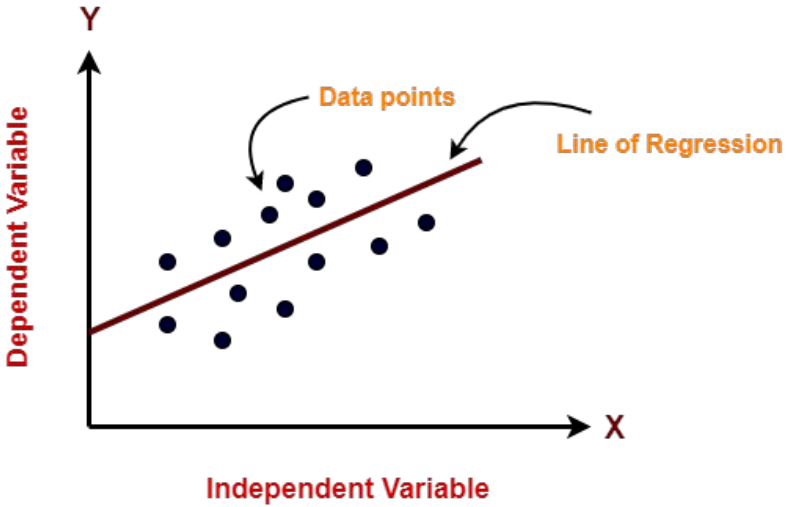


Image source: Analytics Vidhya

Examples of applications of regression

- Annual sales forecast
- Customer satisfaction analysis
- Home price estimation
- Salary estimation

Examples of applications of regression

- Estimating temperatures
- Estimating rainfall intensity
- Life expectancy of a person
- Amount of time a person spends browsing a website

Types of regression models

- Simple regression

  An independent variable is used to predict the dependent variable.

- Multiple regression

  More than one independent variable is used to make the prediction.

Regresión simple vs. múltiple con números

EJEMPLO

Simple: 1 variable independiente

`precio = 1200 · m² + 15 000`

Casa de 80 m^2:
1200 · 80 + 15 000 = **$111 000**

Ignora todo lo demás: dos casas de 80 m^2 siempre costarían lo mismo.

Múltiple: varias variables independientes

`precio = 1000·m² + 8000·hab - 500·años + 20 000`

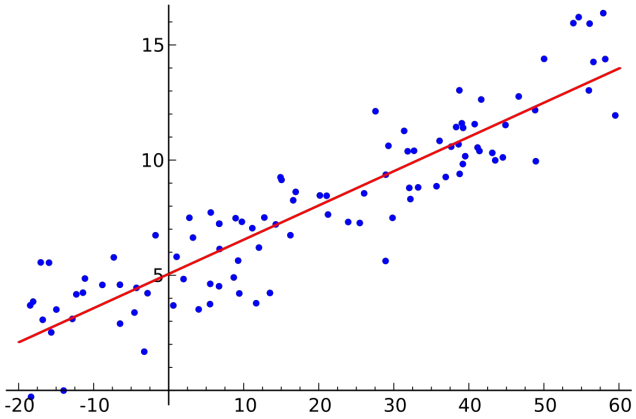
Casa de 80 m^2, 3 habitaciones, 10 años:
80 000 + 24 000 − 5 000 + 20 000 = **$119 000**

Considera más características → suele predecir mejor.

El signo del peso dice cómo influye: **+8000** por habitación sube el precio; **−500** por año de antigüedad lo baja.

Linear regression model

- Linear model that describes the relationship between two or more independent variables.
- The objective of the model is to obtain a straight line that best fits the data.



What is known
x=60 -> y=12
What the model says
x=60 -> y=14

Fuente imagen: Wikipedia

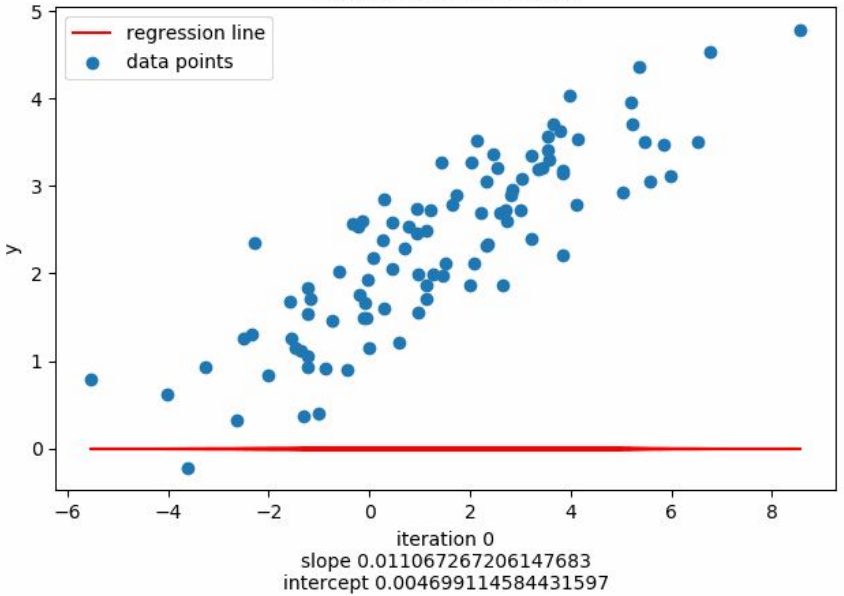
Some important concepts in regression

- Prediction
  - The output of the model
- Weights
  - The result of multiplying each feature with a factor (value)
- Bias
  - A constant value that is not related to features.

Slope and intercept and their relationship with linear regression

Slope – the inclination of a straight line whose value is constant. This value corresponds to the weight associated with a characteristic.

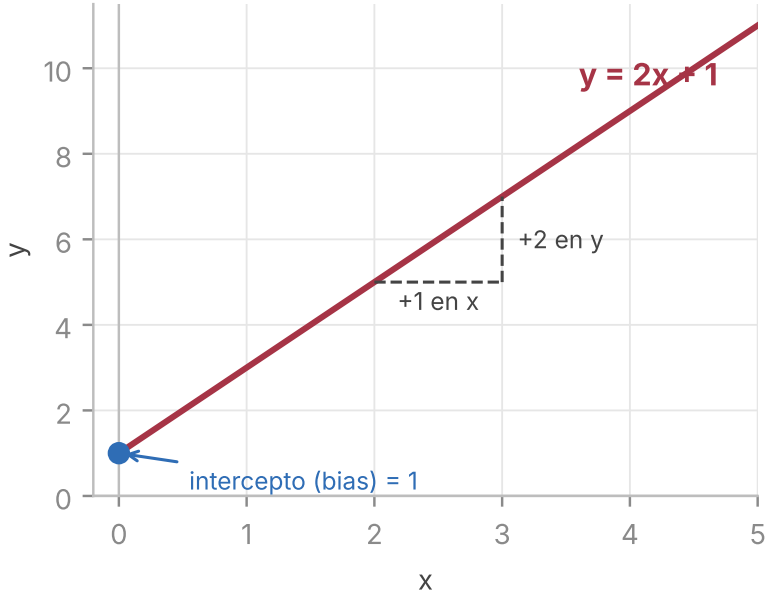
Intercept – the height at which the line crosses the vertical axis. This value corresponds to the bias.



Fuente imagen: machinelearning101

Pendiente (peso) e intercepto (bias) en la práctica

EJEMPLO



Pendiente = 2

Cada vez que x aumenta 1, y aumenta 2.

Intercepto = 1

Cuando x = 0, la recta cruza el eje vertical en y = 1.

En machine learning

`precio = 1200 · m² + 15 000`

Peso = 1200 → cada m^2 extra suma $1 200.

Bias = 15 000 → valor base que no depende de los m^2.

Graphs in Python

- The most known library to create graphs in Python is called matplotlib
- Several types of graphs can be done with the library such as, bar plots, histograms, scatter plots, etc.



Image source: Matplotlib examples

Matplotlib

- Installation

https://matplotlib.org/stable/users/installing/index.html

- Use of the library in Python

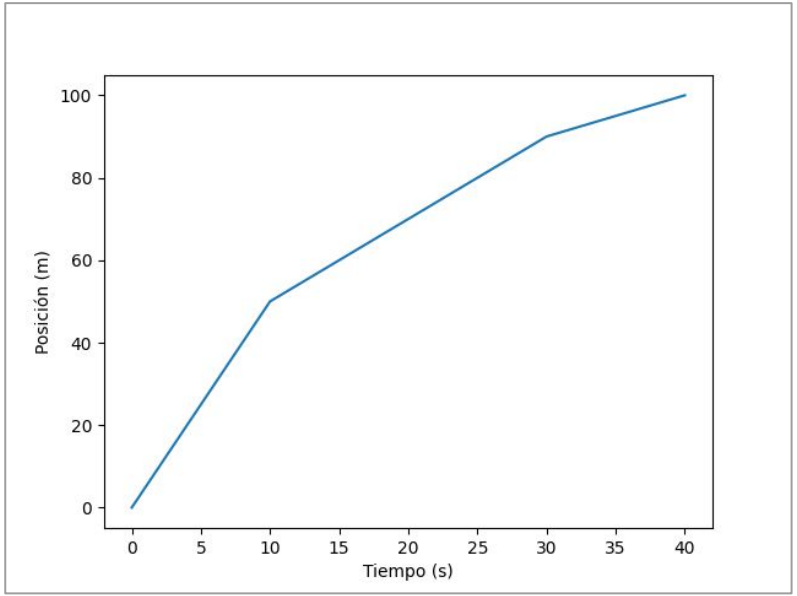
```
import matplotlib.pyplot as plt
```

Line plot in Python

```
import matplotlib.pyplot as plt

time = [0,10,20,30,40]
position = [0,50,70,90,100]

plt.plot(time,position)
plt.xlabel('Time(s)')
plt.ylabel('Position(m)')
plt.show()
```

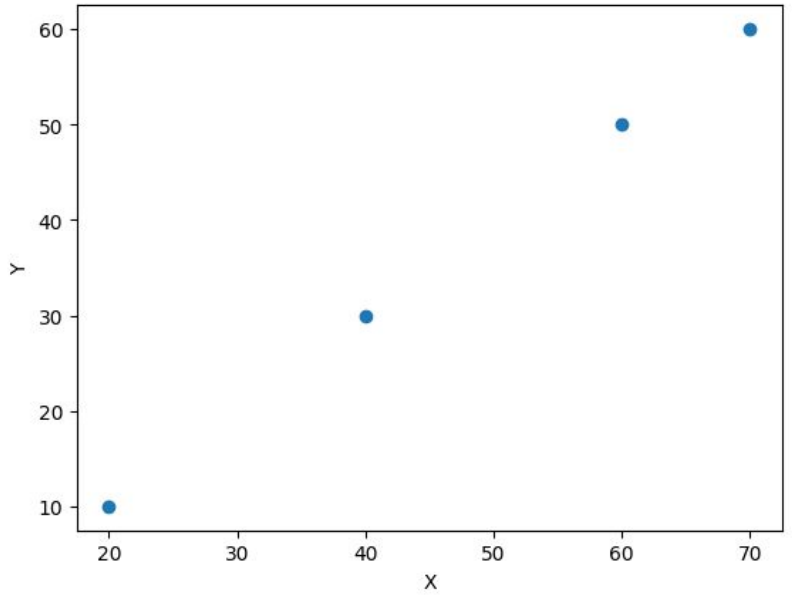


Scatter Plot in Python

```
import matplotlib.pyplot as plt

x= [20,40,60,70]
y = [10,30,50,60]

plt.scatter(x,y)
plt.xlabel("X")
plt.ylabel("Y")
plt.show()
```

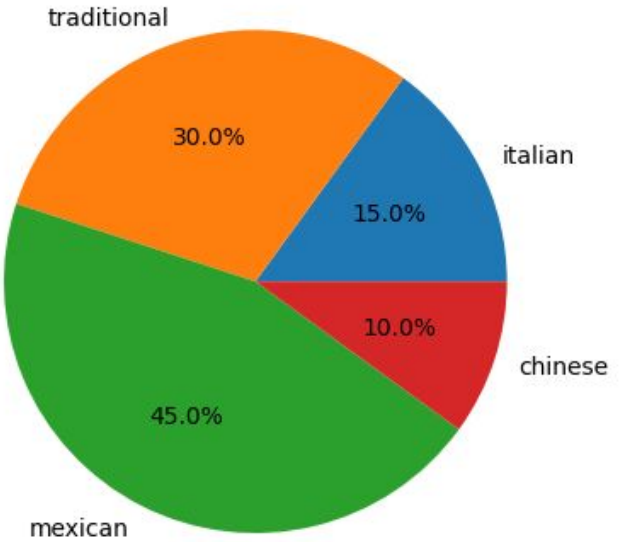


Pie chart in Python

```
import matplotlib.pyplot as plt

food = ['italian','traditional','mexican','chinese']
prefPerc = [15,30,45,10]

plt.pie(prefPerc,labels=food, autopct='%1.1f%%')
plt.show()
```



Matplotlib: scatter + línea de regresión

EJEMPLO

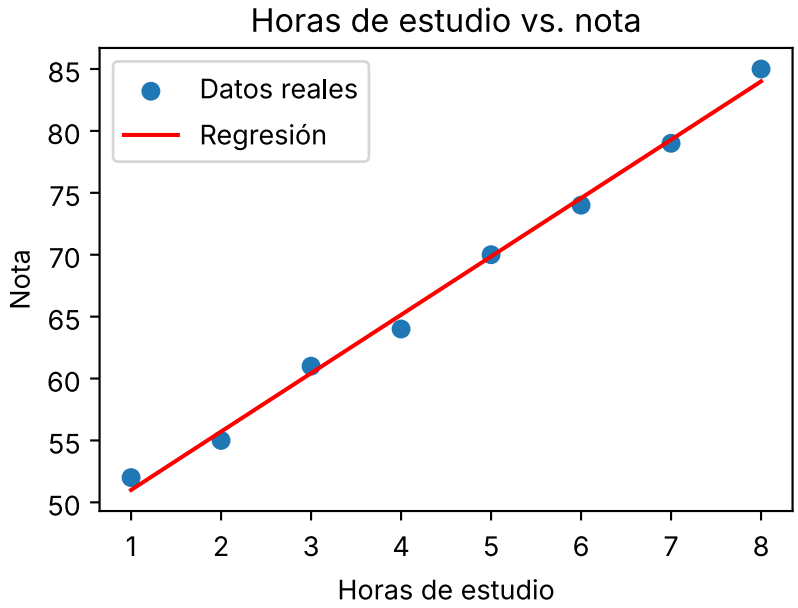
```
[1] import numpy as np
    import matplotlib.pyplot as plt

    horas = np.array([1, 2, 3, 4, 5, 6, 7, 8])
    nota = np.array([52, 55, 61, 64, 70, 74, 79, 85])

    m, b = np.polyfit(horas, nota, 1)  # recta que mejor se ajusta
    print(f"pendiente = {m:.2f}, bias = {b:.2f}")

    plt.scatter(horas, nota, label="Datos reales")
    plt.plot(horas, m * horas + b, color="red", label="Regresión")
    plt.xlabel("Horas de estudio")
    plt.ylabel("Nota")
    plt.title("Horas de estudio vs. nota")
    plt.legend()
    plt.show()
```

```
pendiente = 4.71, bias = 46.29
```



`scatter` dibuja los datos; `plot` la recta. Cada hora extra suma ≈ 4.7 puntos (la pendiente).

Numpy

- Python library used to work with arrays.
- Created in 2005 by Travis Oliphant, as an open source project for free use.
- Numpy stands for Numerical Python.
- Current version: 2.5.3 (September 2026).



Using numpy in Python

- To use numpy, you need to install the library (https://numpy.org/install/).

- Once the library is installed, you import it into the code as follows:

```
import numpy as np
```

Arrays in Numpy

- Numpy stores data by using an array.
- Compared to lists, arrays are faster and contain methods to perform operations.
- There are two types of arrays that can be created in Numpy: vectors (1D) and matrices (2D).

Lista vs. array de NumPy

EJEMPLO

```
[1] import numpy as np
    lista = [1, 2, 3]
    arr = np.array([1, 2, 3])
```

```
[2] lista * 2
```

```
[1, 2, 3, 1, 2, 3]
```

Lista: repite los elementos.

```
[3] print(arr * 2)
    print(arr + 10)
```

```
[2 4 6]
[11 12 13]
```

Array: opera sobre cada elemento (×2 y +10).

```
[4] lista + 10
```

```
TypeError: can only concatenate list (not "int") to list
```

Lista: no se puede sumar un número directamente.

Con un array, las operaciones se aplican **elemento por elemento** sin escribir un for. Por eso es ideal para cálculos con muchos datos.

```
[5] vector = np.array([4, 5, 6])
    matriz = np.array([[1, 2], [3, 4]])
    print(vector.ndim, matriz.ndim)
```

```
1 2
```

ndim = número de dimensiones: **1** → vector (1D), **2** → matriz (2D).

Useful functions in numpy

Function	Description
`np.array()`	Creates a NumPy array from a list or tuple.
`np.zeros()`	Generates an array filled with zeros.
`np.ones()`	Generates an array filled with ones.
`np.arange()`	Creates an array with evenly spaced values within a given range.
`np.linspace()`	Generates an array with evenly spaced numbers between two specified values.

Useful functions in numpy

Function	Description
`np.reshape()`	Changes the shape of an array without changing its data.
`np.ravel()`	Flattens a multi-dimensional array into a 1D array.
`np.transpose()`	Returns the transposed version of an array (swaps rows and columns).

Funciones de NumPy: código y resultado

EJEMPLO

Código	Resultado (print)	Qué pasó
`np.array([1, 2, 3])`	`[1 2 3]`	arreglo a partir de una lista
`np.zeros(3)`	`[0. 0. 0.]`	3 ceros (decimales)
`np.ones((2, 3))`	`[[1. 1. 1.]` ` [1. 1. 1.]]`	2 filas × 3 columnas de unos
`np.arange(0, 10, 2)`	`[0 2 4 6 8]`	de 0 a 10 (sin incluir) de 2 en 2
`np.linspace(0, 1, 5)`	`[0.  0.25 0.5  0.75 1. ]`	5 números igualmente espaciados entre 0 y 1
`a = np.arange(6)`	`[0 1 2 3 4 5]`	0 a 5
`m = np.reshape(a, (2, 3))`	`[[0 1 2]` ` [3 4 5]]`	mismos 6 datos en 2 filas × 3 columnas
`np.ravel(m)`	`[0 1 2 3 4 5]`	aplana: vuelve a 1D

`np.transpose(m)`	`[[0 3]`
	` [1 4]`
	` [2 5]]`
		filas ↔ columnas: ahora 3 × 2

arange elige el **paso**; **linspace** elige **cuántos** números. reshape solo funciona si el total coincide: 6 = 2 × 3.

Numpy array - example

Numpy

```
[7]  import numpy as np
```

Create an array

```
[8]  A = np.array([1,3,5])
```

```
[9]  type(A)
```

```
numpy.ndarray
```

Numpy matrix - example

Create a matrix

```
[10] B = np.array([[1,2,3],[2,4,6]])
```

```
[11] B
```

```
array([[1, 2, 3],
       [2, 4, 6]])
```

```
[12] B.size
```

```
6
```

```
[13] B.ndim
```

```
2
```

```
[14] B.shape
```

```
(2, 3)
```

```
[15] b= B.reshape(3,2)
```

```
[16] b
```

```
array([[1, 2],
       [3, 2],
       [4, 6]])
```

ndim, shape y size: notas de 3 estudiantes en 4 parciales

EJEMPLO

```
[1] B = np.array([[80, 75, 90, 85],
                  [60, 70, 65, 72],
                  [95, 88, 92, 99]])
    B.ndim
```

```
2
```

2 dimensiones: filas y columnas.

```
[2] B.shape
```

```
(3, 4)
```

(filas, columnas) → 3 estudiantes, 4 parciales.

```
[3] B.size
```

```
12
```

```
[4] B.reshape(2, 6)
```

```
array([[80, 75, 90, 85, 60, 70],
       [65, 72, 95, 88, 92, 99]])
```

Los mismos 12 datos en otra forma.

	Parcial 1	Parcial 2	Parcial 3	Parcial 4
Est. 1	80	75	90	85
Est. 2	60	70	65	72
Est. 3	95	88	92	99

shape responde «¿qué forma tiene?»; **size** «¿cuántos datos hay?» (3 × 4 = 12); **ndim** «¿cuántas dimensiones?».

`B.reshape(5, 3)` daría error: 5 × 3 = 15 ≠ 12.

Summary

- Dependent and independent variables
- Matplotlib
- Numpy
- Introduction to regression models

Week 7

Funciones de pérdida y de costo: cómo encontrar la recta que mejor se ajusta

13.10.2026 - 15.10.2026

Learning outcomes

- Describe the difference between loss functions and cost function
- Describe the different metrics for assessing the fit of a model
- Define the concept of series in the context of Pandas
- Describe the steps to create plots with Pandas

How to find the best fit?

- Using a metric to quantify the accuracy of linear regression. This metric is also known as the loss function.
- The loss is the result of the difference between the actual data and the data obtained by the linear regression.

How to find the best fit?

Calculate the error or residual, that is, the difference between the observed value and the value predicted by the model.

Example:

$$x_1 = 20$$

$$y = 10 \rightarrow \text{(observed value)} \qquad \text{(x,y)} - \text{(20,10)}$$

$$\hat{y} = 5 \rightarrow \text{(prediction)}$$

$$error = y - \hat{y} = 5$$

Calcular el error (residuo) punto por punto

EJEMPLO

Modelo: $\hat{y} = 2x + 1$. Para cada dato real se compara el valor observado **y** con la predicción **$\hat{y}$**.

x	y (observado)	$\hat{y}$ (predicción)	error = y − $\hat{y}$	Interpretación
1	5	2·1 + 1 = 3	5 − 3 = **+2**	el modelo se quedó corto
2	4	2·2 + 1 = 5	4 − 5 = **−1**	el modelo se pasó
3	9	2·3 + 1 = 7	9 − 7 = **+2**	el modelo se quedó corto
4	8	2·4 + 1 = 9	8 − 9 = **−1**	el modelo se pasó

Error **positivo**: el punto está **arriba** de la recta. Error **negativo**: está **abajo**.

Ojo: la suma de errores da solo 2 aunque el modelo falló por 6 unidades en total: los positivos y negativos se cancelan. Por eso se usan cuadrados o valores absolutos.

How to find the best fit?

The idea of finding the line that fits the data is that the error obtained between the observed value and the prediction is minimal.

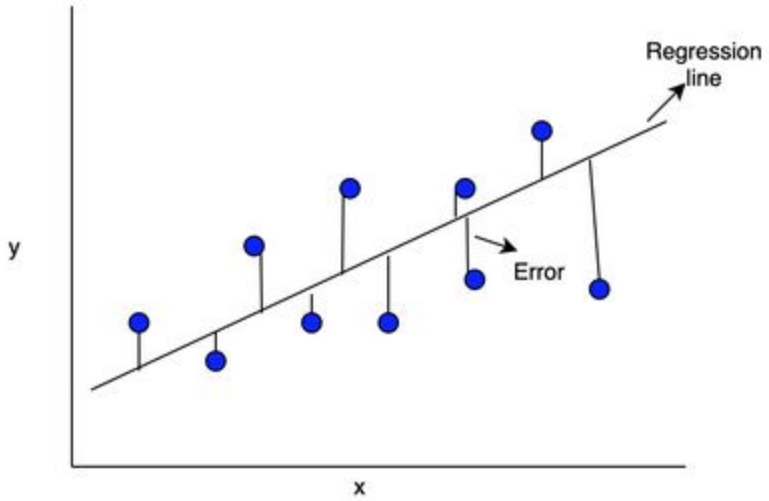
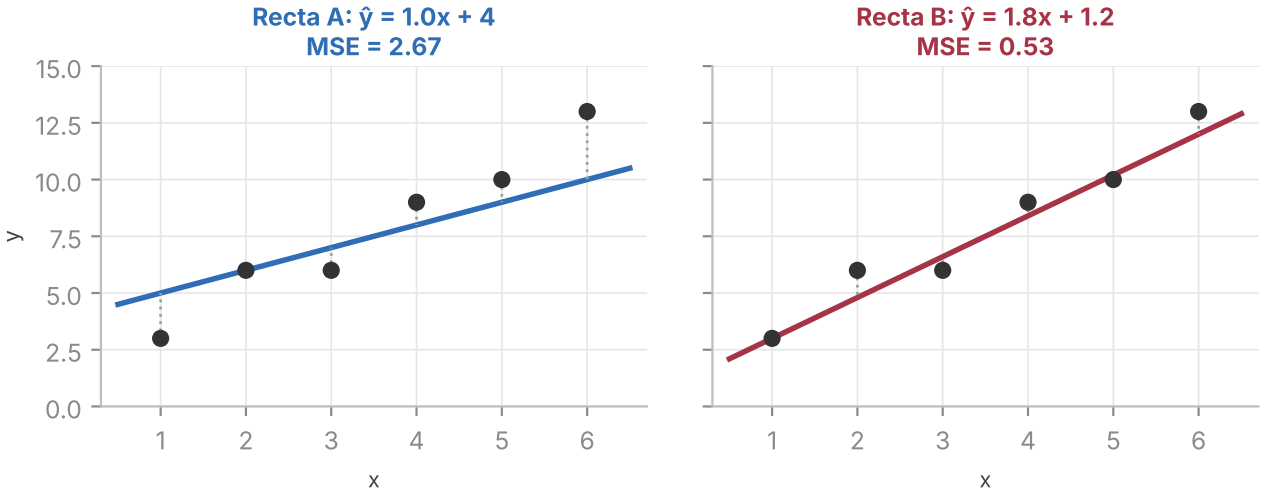


Image source: Cloudera Community

¿Cuál recta se ajusta mejor? La de menor error

EJEMPLO



Las líneas punteadas son los **errores**. La recta A deja puntos muy lejos (hasta 3 unidades); la B pasa cerca de todos.

MSE de A = 2.67 > MSE de B = 0.53 → **B se ajusta mejor**. Entrenar = buscar la pendiente y el bias que hacen este error lo más pequeño posible.

Loss function vs cost function

These two terms are often defined similarly, however there is a particular distinction:

- Loss function. It refers to the error for a single training sample.
- Cost function. It refers to the average of the loss function across the entire training dataset.

Loss vs. cost con los mismos 4 datos

EJEMPLO

Dato (x)	error	loss de ESE dato (error^2)
1	+2	$(+2)^2$ = **4**
2	-1	$(-1)^2$ = **1**
3	+2	$(+2)^2$ = **4**
4	-1	$(-1)^2$ = **1**
		Suma = 10

Loss function → un solo dato

El error de **una** muestra. Ej.: para x = 1 la loss es **4**.

Cost function → todo el dataset

El **promedio** de las losses:
`(4 + 1 + 4 + 1) / 4 = 2.5`
Este promedio de errores al cuadrado se llama **MSE**.

Analogía: la loss es tu nota en **un** deber; el cost es tu **promedio** del parcial.